

REVERSIBLE DATA HIDING BASED ON INTERPOLATION TECHNIQUES

A MINOR PROJECT REPORT

Submitted by

SUNKARI MULA VISHNU VEERANJAN

Under the Guidance of

Dr.C.C.Lin

National Chin-Yi University of Technology

Dr.T.Kavitha

Vel Tech Rangarajan Dr.Sagunthala R&D Institute of Science and Technology

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING



Vel Tech
Rangarajan Dr. Sagunthala
R&D Institute of Science and Technology
(Deemed to be University Estd. u/s 3 of UGC Act, 1956)

DECEMBER 2022



Vel Tech
Rangarajan Dr. Sagunthala
R&D Institute of Science and Technology
(Deemed to be University Estd. u/s 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this minor project report entitled “**REVERSIBLE DATA HIDING BASED ON INTERPOLATION TECHNIQUES** ” is the bonafide work of”**SUNKARI MULA VISHNU VEERANJAN (19UEEC0546)**” who carried out this minor project work under my supervision.

SUPERVISORS

Dr.C.C.Lin

Professor

Department of Computer Science and
Information Engineering

Dr.T.KAVITHA

Professor

Department of ECE

HEAD OF THE DEPARTMENT

Dr.P.ESTHER RANI

Professor

Department of ECE

Submitted for Evaluation of Project Review held on:-----

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our deepest gratitude and sincere thanks to our respected Founder President and Chancellor **Col. Prof. Dr. R. RANGARAJAN** , Foundress President **Dr. SAGUNTHALA RANGARAJAN** , Managing Trustee and Vice President for their immense support in all our endeavours.

We would like to express our gratitude towards our Vice Chancellor **Prof. Dr. S. SALIVAHANAN**, for his kind cooperation and encouragement.

We are obligated to our beloved registrar for **Dr. E. KANNAN**, for providing immense support in all our endeavours.

We are thankful to our esteemed Dean of Academics **Dr. A. T. RAVICHANDRAN**, for providing a wonderful environment to complete our work successfully.

We are extremely thankful and pay our gratitude to our Dean **Dr. V. JAYASANKAR**, for his valuable guidance and support on completion of this project.

It is a great pleasure for us to acknowledge the assistance and contribution of our Head of the Department **Dr. P. ESTHER RANI**, Professor, and Head of Department of Electronics and Communication Engineering for allowing me to undertake this minor project work.

I am grateful to internal my Supervisor **Dr. T.KAVITHA**, Professor, of Electronics and Communication Engineering for her continuous guidance advice effort and inevitable suggestions through out the project work.

I am grateful to my external Supervisor **Dr. CHIA-CHEN LIN**, Professor, Computer Science and Information Engineering, National Chin-Yi University of Technology for her continuous guidance advice effort and inevitable suggestions through out the project work.

SUNKARI MULA VISHNU VEERANJAN

TABLE OF CONTENTS

ABSTRACT	v
LIST OF FIGURES	vi
LIST OF ABBREVIATIONS	vii
1 INTRODUCTION	1
1.1 Image Interpolation	2
1.2 Pixels	3
2 LITERATURE REVIEW	4
3 PROPOSED SCHEME	7
3.1 Pixel Manipulation	7
3.1.1 Data of the image to be hidden	8
3.2 Data Embedding	8
3.3 Data Extraction	12
3.3.1 P1 not equal to P2	13
3.3.2 If P1 equal to P2	17
4 MATLAB CODE	21
4.1 Data Embedding	21
4.2 Data Extraction	32
5 RESULTS AND DISCUSSION	49
5.1 Results	49
5.2 Discussion	52
6 CONCLUSION	53
6.1 Conclusion	53
REFERENCES	54

ABSTRACT

The combination of Internet of Things (IoT) and mobile cloud computing has brought about a standard shift in the service sector. While there are many ways to prevent unauthorized access to information exchanged between mobile phones and cloud platforms, there are no security mechanisms to prevent unauthorized access by cloud administrators. In this project the software is developed in a way that the secret data can be hidden in to a common images found in the internet which is the cover image and also the original image can be recovered later, after extracting the data which is embedded into the cover image this process is known as Reversible data hiding. This Reversible data hiding technique can be improved by using the image interpolation technique. Recently, interpolation techniques are used in the field of information hiding. Interpolation techniques are important in digital image processing and it is worthwhile to use in information hiding to improve the embedding capacity and the visual image quality. when the cover image can be restored together with extracting the secret data at a receiver. where image interpolation techniques are used before embedding the secret data.

Keywords: Interpolation, Reversible Data, Information Hiding

LIST OF FIGURES

3.1	2X2 original block	7
3.2	3X3 original block	8
3.3	Pixel Manipulation	9
3.4	Pixel Manipulation	10
3.5	Pixel Permutation Table	11
3.6	Stego Block	12
5.1	Stego Image	49
5.2	Extracted Original Image	50
5.3	Extracted Secret Image	51

LIST OF ABBREVIATIONS

<i>HVS</i>	HUMAN VISIBLE SCALE
<i>PSNR</i>	PEAK SIGNAL TO NOISE RATIO
<i>SSIM</i>	STRUCTURAL SIMILARITY MEASURE INDEX
<i>BER</i>	BIT ERROR RATE

CHAPTER 1

INTRODUCTION

Though technology is changing our life for better, but it has its own fallouts. Cloud-assisted platforms monitoring system integrates the current promising areas of mobile communications, IoT, and cloud computing technologies to provide feedback, decision support, and is proving to be a revolutionary approach to the security. Unfortunately, such systems subject the data, at privacy and intellectual property risk, which could deter the wide adoption of technology.

Further, with the advancement in technology, the risks of intentional attacks to the data . Last year, on June 14, 2017, Pacific Alliance Medical Center in Los Angles suffered a ransomware attack that resulted in breaching the sensitive information of 266,123 patients (Data Breaches, 2017). A similar patient data security breach was reported by Women’s Health Care Group of Pennsylvania, in July 2017. In this breach, reportedly sensitive EPR of more than 300,000 patients got compromised (Healthcare IT News, 2017). In recent years some renowned healthcare organizations received imposition of imposed hefty fines as a result of tampering with patient data. In June 2016 Catholic Healthcare Services of the Archdiocese of Philadelphia, was slapped with a fine for violating Health Insurance Portability and Accountability Act (HIPAA) security (Healthcare IT News, 2016). Similarly, Mississippi University Medical Center, in July 2016, was fined for EPR violations under HIPAA regulations (Mississippi Today, 2016). A number of similar issues have been reported and could be seen in (Lozupone, 2017; Pussewalage Oleshchuk, 2016; Roumani, Nwankpa, Roumani, 2016; Soomro, Shah, Ahmed, 2016; Sultan, 2014; Wilson, Turban, Zviran, 1992). To avoid such breaches and protect the sensitive information.

Though cryptographic techniques are being used to secure the data to ensure its safe transmission over insecure channels (Ogiela Ogiela, 2012), but, the camouflaged appearance of encrypted data makes the adversary more suspicious and thus invites more attacks. Information hiding, which includes steganography and watermarking, is an alternative technology, which has flourished in the last decade for hiding the secret information imperceptivity in a cover medium (Shehab et al., 2018; Parah, Sheikh, Bhat, 2012). This technique could be used effectively for hiding the data into the

images, thus securely transferring the data to its destination. This is possible only because of the limitations of the HVS that cannot detect the small changes in the pixels of the digital image. For example, if pixel changes from 250 to 252, for HVS the color shade perception of both pixels seems to be same. Further, if we consider a grey scale image as the cover image which generally acts as a host for secret data, the human visual system is unable to detect any change between the stego and the cover image when the PNSR value goes beyond certain threshold, generally 36 dB (Wu Hwang, 2007).

This project gives a reversible security framework, capable of securely embedding data in to images setup to ensure that cloud administrators have no unauthorized access to client data. The scheme makes use of a novel techniques like interpolation and embedding by pixel permutation to embed information reversibly. For the extraction of data from the stego image, the original image is not required and a very high embedding capacity is achieved. One of the main features of the proposed approach is the histogram invariance between cover and stego images. This feature makes the proposed highly resilient to statistical attacks.

1.1 Image Interpolation

Image interpolation occurs when you resize or distort your image from one pixel grid to another. Image resizing is necessary when you need to increase or decrease the total number of pixels, whereas remapping can occur when you are correcting for lens distortion or rotating an image. This process tries to attain a best estimation of a pixel's intensity based on neighboring pixel values on proximity basis. Interpolation is particularly essential when resizing or resampling of image is performed to meet the specifications of the transmission channel or to present the final image with no visual loss. Image interpolation is generally achieved through one of three methods: nearest neighbor, bilinear interpolation, or bicubic interpolation. Since each method has its own merits and challenges, the choice of appropriate method is based on state of affairs.

Interpolation works by using known data to estimate values at unknown points. Image interpolation works in two directions, and tries to achieve a best approximation of a pixel's intensity based on the values at surrounding pixels. Common interpolation algorithms can be grouped into two categories: adaptive and non-adaptive. Adaptive methods change depending on what they are interpolating, whereas non-adaptive methods treat all pixels equally. Non-adaptive algorithms include: nearest neighbor, bilinear, bicubic, spline, sinc, lanczos and others.

Here we used the nearest neighbour interpolation method where the output pixel value is estimated by considering the nearest pixel's value to the specified input coordinates. This method is simple and easy to implement and no artificial data is introduced in the final output. However the output image contains blocking artifacts which may have effect on error calculations in performance evaluation.

1.2 Pixels

Images are made of picture elements called pixels. Typically, pixels are organized in an ordered rectangular array. The size of an image is determined by the dimensions of this pixel array. The image width is the number of columns, and the image height is the number of rows in the array. Thus the pixel array is a matrix of M columns $\times$ N rows. To refer to a specific pixel within the image matrix, we define its coordinate at x and y . The coordinate system of image matrices defines x as increasing from left to right and y as increasing from top to bottom. Compared to normal mathematic convention, the origin is in the top left corner and the y coordinate is flipped.

Having defined the number of pixels, $M \times N$, only provides a rectangular shape for our image. One more parameter, intensity, is needed to truly define an image. Each pixel has its own intensity value, or brightness. If all the pixels have the same value, the image will be a uniform shade; all black, white, gray, or some other shade. It is in the type of intensity used for each pixel that image types vary. Black and white images only have intensity from the darkest gray (black) to lightest gray (white). Color images, on the other hand, have intensity from the darkest and lightest of three different colors, Red, Green, and Blue. The various mixtures of these color intensities produces a color image. Thus the two most basic types of digital images, BW and Color, are known as grayscale and RGB images. In addition to the intensity type of each pixel, the range of intensity values also varies.

CHAPTER 2

LITERATURE REVIEW

Zhang, Xianquan, et al.[1] proposed algorithm creates an interpolated image from input image by parabolic interpolation, and embeds secret bits into interpolated pixels in terms of the relation between the interpolated value and the mean value. Ten standard benchmark images are taken as test images for validating efficiency of our algorithm. The results illustrate that our algorithm has better performances than some popular data hiding methods in embedding capacity and visual quality with respect to PSNR and SSIM.

Jung, K. H., Yoo, K. Y. et al. [2] proposed data hiding method is based on interpolation. Comparison on data hiding methods is divided into reversible or not. This paper experimental results show the proposed method can embed a large amount of secret data while keeping a very high visual quality, the PSNR is guaranteed to be higher than 35 dB compared with other reversible data hiding methods. And also capacity is larger than any other reversible data hiding methods and comparable to other data hiding methods.

Jung, K. H. (2018). et al. [3] explained reversible data hiding methods using interpolation techniques are described and analyzed on the embedding capacity and the visual image quality that many researchers have tried to improve these different measurements. It is concluded with the directions of research with some recommendations.

Lee, C. F., Huang, Y. L. et al. [4] studied uses the good results of redundancy from an effective image interpolation to maximize the difference values between neighboring pixels for enhancing embedding capacity to the largest.

Wang, Xing-Tian, Chin-Chen Chang, Thai-Son Nguyen, and Ming-Chu Li. et al. [5] proposed a novel reversible data hiding scheme for high quality images. All pixels of the cover image are classified as wall pixels and non-wall pixels. For wall pixel, the interpolation error is used to embed secret data. For non-wall pixel, we compute the difference value of adjacent pixels based on the direction order to hide secret data. The experimental results demonstrate the proposed scheme provides a better embedding performance than some compared schemes..

Hassan, F. S., Gutub, A. et al. [6] proposed an interpolation-based RDH (IRDH) scheme that improves Lee and Huang's scheme and Malik et al.'s scheme by combining their embedding techniques along with the optimal pixel adjustment process (OPAP) in a way that increases the embedding capacity and the visual quality of both the schemes. In this presented scheme, we start by stretching the size of the original image using the existing enhanced neighbor mean interpolation (ENMI) interpolation technique then the data is embedded into the interpolated pixels using our novel embedding method that depends on the intensity of the pixels and the maximized difference values.

Wahed, M. A., Nyeem, H. et al. [7] A new Interpolation based Reversible Data Hiding (IRDH) scheme is reported in this paper. For different applications of an IRDH scheme to the digital image, video, multimedia, big-data and biological data, the embedding capacity requirement usually varies. Disregarding this important consideration, existing IRDH schemes do not offer a better embedding rate-distortion performance for varying size payloads. To attain this varying capacity requirement with our proposed adaptive embedding, we formulate a capacity control parameter and propose to utilize it to determine a minimum set of embeddable bits in a pixel.

Malik, A., Sikka, G., Verma, H. K. et al. [8] proposed hiding scheme first divides pixel intensity ranges into groups and then adaptively embeds the secret data bits into the pixels based on the pixel intensity values. Therefore, the proposed scheme is able to maintain the visual quality of the stego-image. Experimental results show that the achieved PSNR by the proposed interpolation method is more than 30 dB for all the test images. Further, the results prove that the proposed data hiding scheme has superior performance than all the existing interpolation-based data hiding schemes.

Malik, A., Sikka, G., Verma, H. K.. [9] proposed data hiding scheme comes into the category of reversible data hiding scheme as the input image can be reconstructed after extraction of the entire secret data at the receiver side. Thus, it reduces the communication cost. Furthermore, the proposed data hiding scheme identifies the smooth and complex regions of the interpolated (or cover) image by dividing the same into blocks. It then embeds more bits into the complex regions of the image so that data hiding capacity as well as the image quality can be enhanced. The experimental results shows that the percentage increment in the PSNR value and capacity of the proposed scheme with respect to Chang et al. method is in the range of 0.26 to 30.60 percent and 0.87 to 73.82 percent, respectively.

Moreover, the modified NMI yields higher PSNRs than other interpolating methods such as NMI, BI, and ENMI.

Shaik, A., et al. [10] analysed that the proposed 2D parabolic interpolation maximized the utilization of the cover image pixels in up-sampling. The technique produced interpolated images of a higher quality compared to the 1D parabolic interpolation technique. Moreover, the proposed data hiding technique exploited the local redundancy and obtained the high embedding capacity with an appreciable image quality better than other state-of-the-art techniques. The results of this study support the view that the proposed parabolic interpolation has the potential to address security issues in high embedding capacity applications..

Meikap, S., Jana, et al. [11] proposed a generalized directional PVO (DPVO) with varying block size. The original image is partitioned into blocks and then enlarged using image interpolation. A new parameter α is introduced and added with maximum pixel value and subtracted from minimum pixel value to maintain the order of the rank which is dependent on the size of the image block. To improve data hiding capacity, overlapped embedding has been considered in three different directions (1) Horizontal, (2) Vertical and (3) Diagonal of each block.

Govind, P. S., Wilsy, M. et al. [12] proposed a further modification to the scheme of Huang et al. based on directional interpolation. Directional interpolation yields a better approximation to the original pixel which improves the capacity of embedding. The effectiveness of the proposed scheme is tested using standard test images and the proposed scheme gives better results in terms of embedding capacity and visual quality compared to Huang et al. scheme.

CHAPTER 3

PROPOSED SCHEME

The proposed algorithm is divided in detail into three phases. In first phase the original image is scaled down and interpolated (denoted as stego image) in the algorithm as discussed in detail in section 3.1 and the second phase is embedding of the data in to the selected image using pixel permutation and in the second one the extraction of the data from the data embedded image stego image and in the last phase the data is extracted from the stego image while as section 3.3 presents the data extraction process.

3.1 Pixel Manipulation

To ensure reversibility and carry out embedding based on the permutation table shown in table 1, the input image is subjected to Pixel manipulation. The pixel manipulation is done in way that let assume that 'P' represents the original image. The matrix block of the pixels of the original image is divided into 2×2 blocks that are denoted as (P1,P2,P3,P4) so that the blocks can be interpolated and converted as 3×3 blocks that is explained in the following equations.

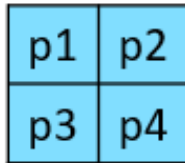


Figure 3.1: 2X2 original block

- (1) $ep1 = P1 + 1;$
- (2) $ep2 = P1 - 1;$
- (3) $ep3 = P1 + 2;$
- (4) $ep4 = P2 + 1;$
- (5) $ep5 = P4 - 1;$

The "ep1,ep2,ep3,ep4,ep5" are positioned as shown in the figure below:

p1	ep1	P2
ep2	ep3	ep4
p3	ep5	p4

Figure 3.2: 3X3 original block

3.1.1 Data of the image to be hidden

The secret image is resized into 256×256 pixels so that it can be fit into the interpolated image and the image is converted into binary data that is 0's and 1's and manipulated the matrix into one row so that the pixels can be divided into every 6bits that will be the combination of 0's and 1's. This 6 bit data is further divided in to 5bit and 1bit format to produce the hiding algorithm more complicate that can't be theft by the attackers.

3.2 Data Embedding

In this section we discuss the data embedding based on the 5bit and 1bit data extracted from the secret image detail in section 3.1.1.This section is divided in to two phases,the first phase is data manipulation in the cover block based on the 6th bit extracted and the other is pixel position changing based on the 5bit data which is extracted comparing to the permutation table.

In the first phase the first and second elements in the original block are equal i.e., $P1=P2$ and if the 6th bit extracted from the secret bit is 0 the following equations are followed for pixel manipulation

- (1) $ep1=P1+1;$
- (2) $ep2=P3+1;$
- (3) $ep3=P3+2;$
- (4) $ep4=P4+1;$
- (5) $ep5=P3-1;$

If the 6th bit of secret data is 1 then,

(1) $ep1 = P2 + 1$;

(2) $ep2 = P3 + 1$;

(3) $ep3 = P4 + 2$;

(4) $ep4 = P4 + 1$;

(5) $ep5 = P4 - 1$;

The example for the above explanation is shown in the figure 3.3

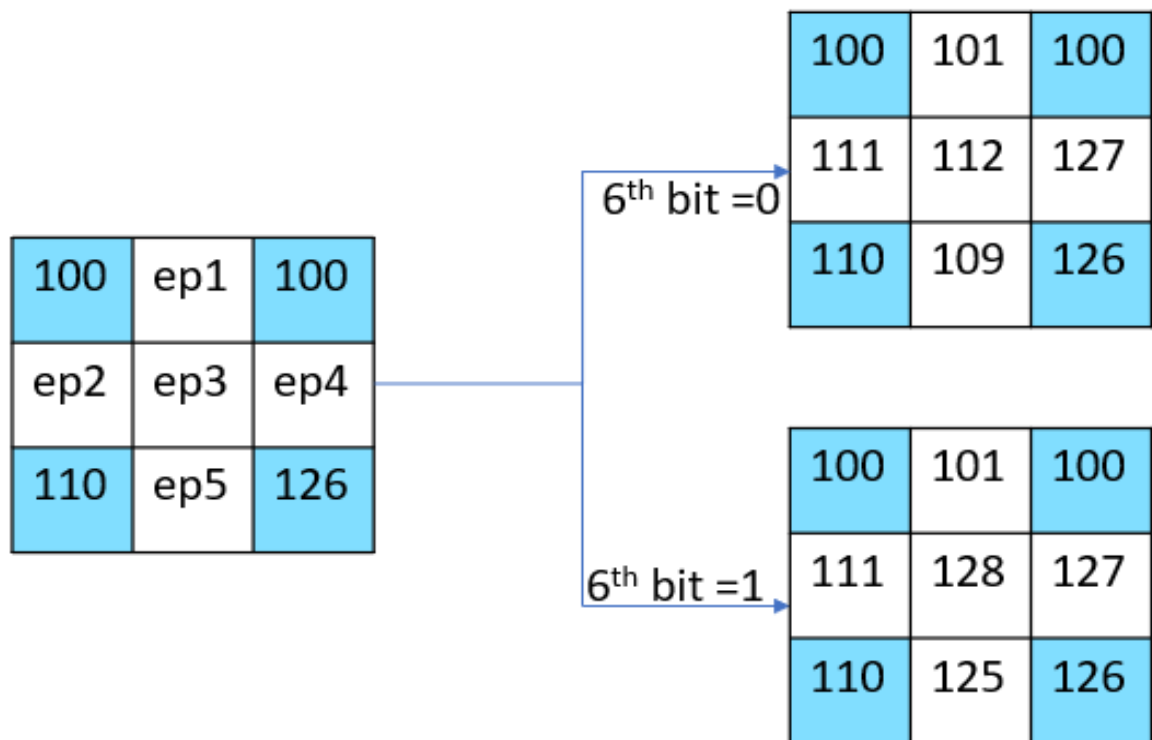


Figure 3.3: Pixel Manipulation

If the first and second elements of the original block are different i.e., $P1 \neq P2$ and if the 6th bit extracted from the secret bit is 0 the following equations are followed for pixel manipulation

- (1) $ep1 = P1 + 1;$
- (2) $ep2 = P1 - 1;$
- (3) $ep3 = P1 + 2;$
- (4) $ep4 = P4 + 1;$
- (5) $ep5 = P4 - 1;$

If the 6th bit of secret data is 1 then,

- (1) $ep1 = P2 + 1;$
- (2) $ep2 = P2 - 1;$
- (3) $ep3 = P2 + 2;$
- (4) $ep4 = P4 + 1;$
- (5) $ep5 = P4 - 1;$

The example for the above explanation is shown in the figure 3.4

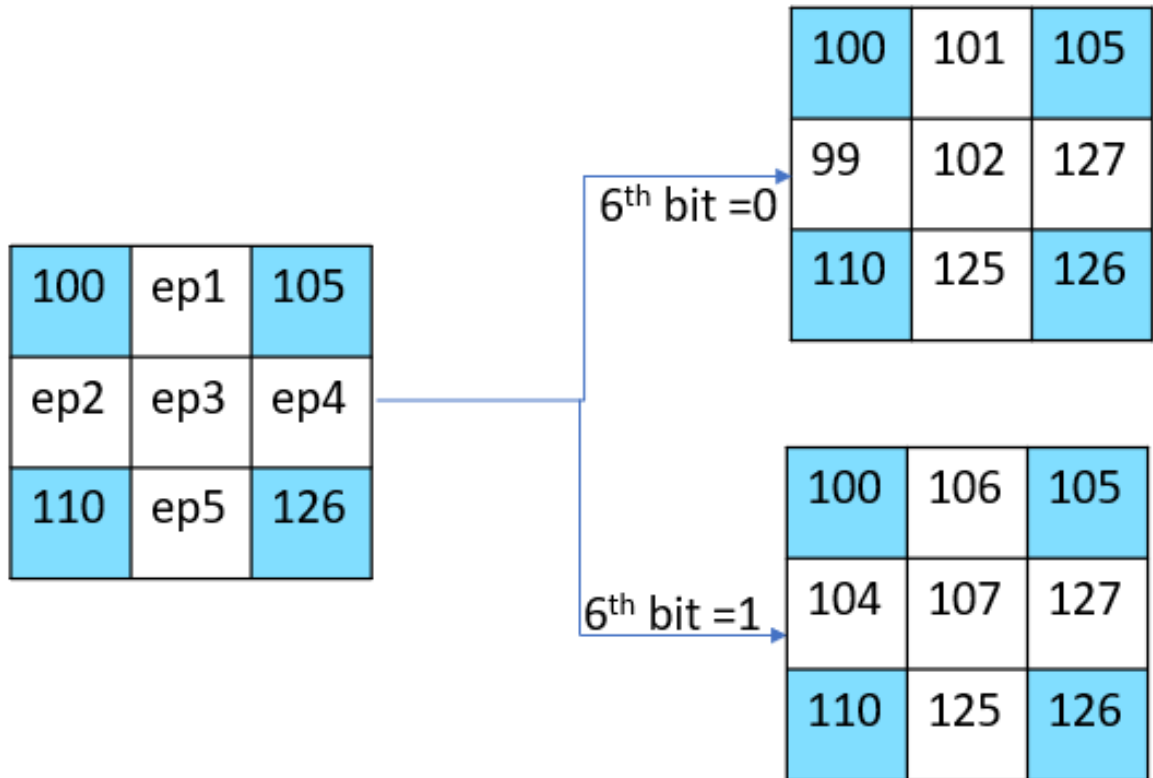


Figure 3.4: Pixel Manipulation

The resulted matrix blocks after the manipulation compared to 6th bit of secret data are denoted as Pixel Position Changed Block (PPCB).

The second phase explains the pixel position changing in the PPCB based the 5bit extracted secret data according to the pixel permutation table,the pixel permutation table is prepared and organised in the way as shown in the figure3.5

Secret Data	Pixel Permuted Block				
	Ep1	Ep2	Ep3	Ep4	Ep5
00000	Ep1	Ep2	Ep3	Ep4	Ep5
00001	Ep1	Ep2	Ep3	Ep5	Ep4
00010	Ep2	Ep1	Ep4	Ep3	Ep5
00011	Ep2	Ep1	Ep5	Ep4	Ep3
00100	Ep2	Ep3	Ep1	Ep4	Ep5
00101	Ep2	Ep3	Ep1	Ep5	Ep4
00110	Ep3	Ep2	Ep5	Ep1	Ep4
00111	Ep3	Ep2	Ep4	Ep5	Ep1
01000	Ep1	Ep5	Ep2	Ep3	Ep4
01001	Ep1	Ep5	Ep3	Ep4	Ep2
01010	Ep5	Ep1	Ep2	Ep3	Ep4
01011	Ep5	Ep1	Ep3	Ep2	Ep4
01100	Ep5	Ep3	Ep2	Ep4	Ep1
01101	Ep5	Ep3	Ep4	Ep2	Ep1
01110	Ep3	Ep5	Ep1	Ep2	Ep4
01111	Ep3	Ep5	Ep2	Ep4	Ep1

Secret Data	Pixel Permuted Block				
	Ep1	Ep2	Ep3	Ep4	Ep5
10000	Ep1	Ep3	Ep4	Ep2	Ep5
10001	Ep1	Ep3	Ep5	Ep2	Ep4
10010	Ep2	Ep4	Ep3	Ep1	Ep5
10011	Ep2	Ep4	Ep1	Ep3	Ep5
10100	Ep2	Ep5	Ep1	Ep4	Ep3
10101	Ep2	Ep5	Ep4	Ep3	Ep1
10110	Ep3	Ep1	Ep2	Ep4	Ep5
10111	Ep3	Ep1	Ep5	Ep2	Ep4
11000	Ep1	Ep4	Ep5	Ep2	Ep3
11001	Ep1	Ep4	Ep3	Ep5	Ep2
11010	Ep5	Ep2	Ep4	Ep1	Ep3
11011	Ep5	Ep2	Ep3	Ep4	Ep1
11100	Ep5	Ep4	Ep2	Ep3	Ep1
11101	Ep5	Ep4	Ep3	Ep1	Ep2
11110	Ep3	Ep4	Ep5	Ep2	Ep1
11111	Ep3	Ep4	Ep2	Ep5	Ep1

Figure 3.5: Pixel Permutation Table

After comparing to the 5bit data the pixels position is changed as per the above figure 3.5 the new 3×3 block is formed as shown in the figure 3.6 denoted as stego block

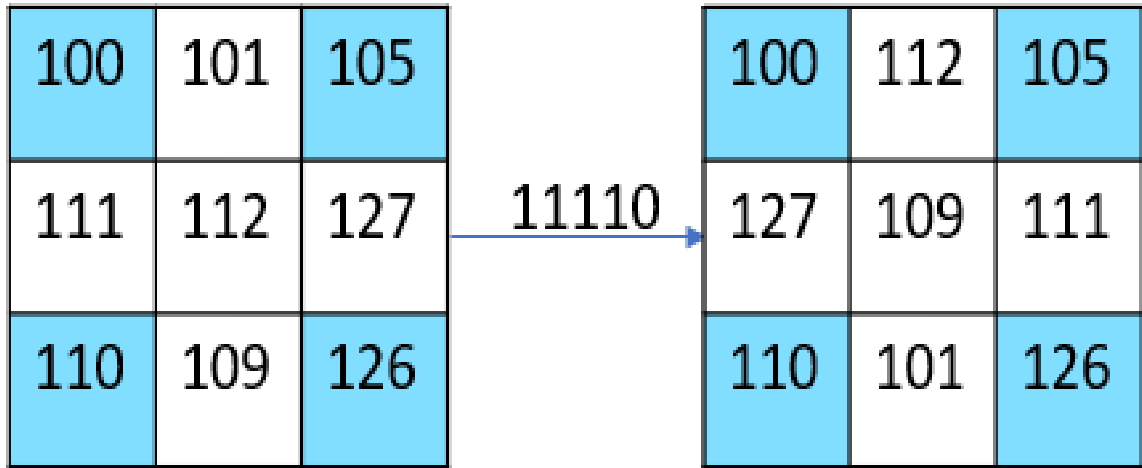


Figure 3.6: Stego Block

The stego blocks which are the final blocks of the image forms the stego image containing the secret image that will not be visible.

3.3 Data Extraction

The stego image is processed to get the pixels of the image and that will be divided into 3×3 blocks. The 2×2 original can be restored from the extracted blocks as the $(1,1), (1,3), (3,1),$ and $(3,3)$ are not changed in the process of pixel manipulation from the cover block. Here we have focus on that the $P1, P2, P3,$ and $P4$ are not changed through out the algorithm.

Comparing the stego image blocks to the restored original blocks the secret image can be extracted by using the following conditions,

3.3.1 P1 not equal to P2

STEGO BLOCK					SECRET DATA
Ep1	Ep2	Ep3	Ep4	Ep5	
P1+1	P1-1	P1+2	P4+1	P4-1	000000
P2+1	P2-1	P2+2	P4+1	P4-1	000001
P1+1	P1-1	P1+2	P4-1	P4+1	000010
P2+1	P2-1	P2+2	P4-1	P4+1	000011
P1-1	P1+1	P4+1	P1+2	P4-1	00010
P2-1	P2+1	P4+1	P2+2	P4-1	000101
P1-1	P1+1	P4-1	P4+1	P1+2	000110
P2-1	P2+1	P4-1	P4+1	P2+2	000111
P1-1	P1+2	P1+1	P4+1	P4-1	001000
P2-1	P2+2	P2+1	P4+1	P4-1	001001
P1-1	P1+2	P1+1	P4-1	P4+1	001010
P2-1	P2+2	P2+1	P4-1	P4+1	001011
P1+2	P1-1	P4-1	P1+1	P4+1	001100
P2+2	P2-1	P4-1	P2+1	P4+1	001101
P1+2	P1-1	P4+1	P4-1	P1+1	001110
P2+2	P2-1	P4+1	P4-1	P2+1	001111

STEGO BLOCK					SECRET DATA
Ep1	Ep2	Ep3	Ep4	Ep5	
P1+1	P4-1	P1-1	P1+2	P4+1	010000
P2+1	P4-1	P2-1	P2+2	P4+1	010001
P1+1	P4-1	P1+2	P4+1	P1-1	010010
P2+1	P4-1	P2+2	P4+1	P2-1	010011
P4-1	P1+1	P1-1	P1+2	P4+1	010100
P4-1	P2+1	P2-1	P2+2	P4+1	010101
P4-1	P1+1	P1+2	P1-1	P4+1	010110
P4-1	P2+1	P2+2	P2-1	P4+1	010111
P4-1	P1+2	P1-1	P4+1	P1+1	011000
P4-1	P2+2	P2-1	P4+1	P2+1	011001
P4-1	P1+2	P4+1	P1-1	P1+1	011010
P4-1	P2+2	P4+1	P2-1	P2+1	011011
P1+2	P4-1	P1+1	P1-1	P4+1	011100
P2+2	P4-1	P2+1	P2-1	P4+1	011101
P1+2	P4-1	P1-1	P4+1	P1+1	011110
P2+2	P4-1	P2-1	P4+1	P2+1	011111

STEGO BLOCK					SECRET DATA
Ep1	Ep2	Ep3	Ep4	Ep5	
P1+1	P1+2	P4+1	P1-1	P4-1	100000
P2+1	P2+2	P4+1	P2-1	P4-1	100001
P1+1	P1+2	P4-1	P1-1	P4+1	100010
P2+1	P2+2	P4-1	P2-1	P4+1	100011
P1-1	P4+1	P1+2	P1+1	P4-1	100100
P2-1	P4+1	P2+2	P2+1	P4-1	100101
P1-1	P4+1	P1+1	P1+2	P4-1	100110
P2-1	P4+1	P2+1	P2+2	P4-1	100111
P1-1	P4-1	P1+1	P4+1	P1+2	101000
P2-1	P4-1	P2+1	P4+1	P2+2	101001
P1-1	P4-1	P4+1	P1+2	P1+1	101010
P2-1	P4-1	P4+1	P2+2	P2+1	101011
P1+2	P1+1	P1-1	P4+1	P4-1	101100
P2+2	P2+1	P2-1	P4+1	P4-1	101101
P1+2	P1+1	P4-1	P1-1	P4+1	101110
P2+2	P2+1	P4-1	P2-1	P4+1	101111

STEGO BLOCK					SECRET DATA
Ep1	Ep2	Ep3	Ep4	Ep5	
P1+1	P4+1	P4-1	P1-1	P1+2	110000
P2+1	P4+1	P4-1	P2-1	P2+2	110001
P1+1	P4+1	P1+2	P4-1	P1-1	110010
P2+1	P4+1	P2+2	P4-1	P2-1	110011
P4-1	P1-1	P4+1	P1+1	P1+2	110100
P4-1	P2-1	P4+1	P2+1	P2+2	110101
P4-1	P1-1	P1+2	P4+1	P1+1	110110
P4-1	P2-1	P2+2	P4+1	P2+1	110111
P4-1	P4+1	P1-1	P1+2	P1+1	111000
P4-1	P4+1	P2-1	P2+2	P2+1	111001
P4-1	P4+1	P1+2	P1+1	P1-1	111010
P4-1	P4+1	P2+1	P2+1	P2-1	111011
P1+2	P4+1	P4-1	P1-1	P1+1	111100
P2+2	P4+1	P4-1	P2-1	P2+1	111101
P1+2	P4+1	P1-1	P4-1	P1+1	111110
P2+2	P4+1	P2-1	P4-1	P2+1	111111

3.3.2 If P1 equal to P2

STEGO BLOCK					SECRET DATA
Ep1	Ep2	Ep3	Ep4	Ep5	
P1+1	P3+1	P3+2	P4+1	P3-1	000000
P2+1	P3+1	P4+2	P4+1	P4-1	000001
P1+1	P3+1	P3+2	P3-1	P4+1	000010
P2+1	P3+1	P4+2	P4-1	P4+1	000011
P3+1	P1+1	P4+1	P3+2	P3-1	000100
P3+1	P2+1	P4+1	P4+2	P4-1	000101
P3+1	P1+1	P3-1	P4+1	P3+2	000110
P3+1	P2+1	P4-1	P4+1	P4+2	000111
P3+1	P3+2	P4+1	P4+1	P3-1	001000
P3+1	P4+2	P2+1	P4+1	P4-1	001001
P3+1	P3+2	P3-1	P3-1	P4+1	001010
P3+1	P4+2	P2+1	P4-1	P4+1	001011
P3+2	P3+1	P1+1	P1+1	P4+1	001100
P4+2	P3+1	P4-1	P2+1	P4+1	001101
P3+2	P3+1	P3-1	P3-1	P1+1	001110
P4+2	P3+1	P4+1	P4-1	P2+1	001111

STEGO BLOCK					SECRET DATA
Ep1	Ep2	Ep3	Ep4	Ep5	
P1+1	P3-1	P3+1	P3+2	P4+1	010000
P2+1	P4-1	P3+1	P4+2	P4+1	010001
P1+1	P3-1	P3+2	P4+1	P3+1	010010
P2+1	P4-1	P4+2	P4+1	P3+1	010011
P3-1	P1+1	P3+1	P3+2	P4+1	010100
P4-1	P2+1	P3+1	P4+2	P4+1	010101
P3-1	P1+1	P3+2	P3+1	P4+1	010110
P4-1	P2+1	P4+2	P3+1	P4+1	010111
P3-1	P3+2	P3+1	P4+1	P1+1	011000
P4-1	P4+2	P3+1	P4+1	P2+1	011001
P3-1	P3+2	P4+1	P3+1	P1+1	011010
P4-1	P4+2	P4+1	P3+1	P2+1	011011
P3+2	P3-1	P1+1	P3+1	P4+1	011100
P4+2	P4-1	P2+1	P3+1	P4+1	011101
P3+2	P3-1	P3+1	P4+1	P1+1	011110
P4+2	P4-1	P3+1	P4+1	P2+1	011111

STEGO BLOCK					SECRET DATA
Ep1	Ep2	Ep3	Ep4	Ep5	
P1+1	P3+2	P4+1	P3+1	P3-1	100000
P2+1	P4+2	P4+1	P3+1	P4-1	100001
P1+1	P3+2	P3-1	P3+1	P4+1	100010
P2+1	P4+2	P4-1	P3+1	P4+1	100011
P3+1	P4+1	P3+2	P1+1	P3-1	100100
P3+1	P4+1	P4+2	P2+1	P4-1	100101
P3+1	P4+1	P3+2	P1+1	P3-1	100110
P3+1	P4+1	P2+1	P4+2	P4-1	100111
P3+1	P3-1	P1+1	P4+1	P3+2	101000
P3+1	P4-1	P2+1	P4+1	P4+2	101001
P3+1	P3-1	P4+1	P3+2	P1+1	101010
P3+1	P4-1	P4+1	P4+2	P2+1	101011
P3+2	P1+1	P3+1	P4+1	P3-1	101100
P4+2	P2+1	P3+1	P4+1	P4-1	101101
P3+2	P1+1	P3-1	P3+1	P4+1	101110
P4+2	P2+1	P4-1	P3+1	P4+1	101111

STEGO BLOCK					SECRET DATA
Ep1	Ep2	Ep3	Ep4	Ep5	
P1+1	P4+1	P3-1	P3+1	P3+2	110000
P2+1	P4+1	P4-1	P3+1	P4+2	110001
P1+1	P4+1	P3+2	P3-1	P3+1	110010
P2+1	P4+1	P4+2	P4-1	P3+1	110011
P3-1	P3+1	P4+1	P1+1	P3+2	110100
P4-1	P3+1	P4+1	P2+1	P4+2	110101
P3-1	P3+1	P3+2	P4+1	P1+1	110110
P4-1	P3+1	P4+2	P4+1	P2+1	110111
P3-1	P4+1	P3+1	P3+2	P1+1	111000
P4-1	P4+1	P3+1	P4+2	P2+1	111001
P3-1	P4+1	P3+2	P1+1	P3+1	111010
P4-1	P4+1	P4+2	P2+1	P3+1	111011
P3+2	P4+1	P3-1	P3+1	P1+1	111100
P4+2	P4+1	P4-1	P3+1	P2+1	111101
P3+2	P4+1	P3+1	P3-1	P1+1	111110
P4+2	P4+1	P3+1	P4-1	P2+1	111111

CHAPTER 4

MATLAB CODE

4.1 Data Embedding

```
1 A= imread("C:\Users\91863\Desktop\testing\lena.bmp");
2
3 s=size(A);
4 f=2;
5 s1=s/f;
6 k=1;
7 l=1;
8 for i=1:s1
9     for j=1:s1
10        B(i,j) = A(k,l);
11        l=l+f;
12    end
13    l=1;
14    k=k+2;
15 end
16
17 f1=2;
18 s2=s1*f1;
19 k=1;
20 l=1;
21 for (i=1:2:s2)
22     for (j=1:f1:s2)
23        C(i,j)=B(k,l);
24        l=l+1;
25    end
26    l=1;
27    k=k+1;
28 end
29
30 for (i=1:f1:s2)
31     for (j=2:f1:s2)
```

```

32         C(i , j )=0;
33     end
34 end
35
36 for ( j=1:f1 : s2)
37     for ( i=2:f1 : s2)
38         C(i , j )=0;
39     end
40 end
41
42 for ( j=2:f1 : s2)
43     for ( i=2:f1 : s2)
44         C(i , j )=0;
45     end
46 end
47
48 C( : , 4:4 : end) = [];
49 C(4:4 : end , : ) = [];
50
51 [r , c]=size (C) ;
52 rs=3;
53 cs=3;
54 nr=floor ( r / rs ) ;
55 nc=floor ( r / cs ) ;
56 sA(rs , cs , nr*nc)=0;
57 z=1;
58 for i=1:nr
59     for j=1:nc
60         sA( : , : , z) = C(((i-1)*rs+1):rs*i , ((j-1)*cs+1):cs*j) ;
61         z=z+1;
62     end
63 end
64 cc=sA;
65 for j=1:16384
66     sA(1 , 2 , j)=sA(1 , 1 , j)+1;
67     sA(2 , 1 , j)=sA(1 , 1 , j)-1;
68     sA(2 , 2 , j)=sA(1 , 1 , j)+2;
69     sA(2 , 3 , j)=sA(1 , 3 , j)+1;
70     sA(3 , 2 , j)=sA(3 , 3 , j)-1;
71     j=j+1;
72 end
73
74
75 img=imread ("C:\Users\91863\Desktop\logores.bmp" );
76 img=uint8 (img) ;
77
78 img = imresize (img , [256 256]);
79

```

```

80 pic_bin=imbinarize(img);
81 figure ,imshow(pic_bin)
82 row_vec=reshape(pic_bin',1,[]);
83
84 a=row_vec;
85
86 ttt = a(:,6:6:end);
87 bit6= reshape(ttt,1,1,[]);
88 a(6:6:end)=[];
89 a(1,54615)=0;
90 temp= reshape(a,1,5,[]);
91
92
93 for z=1:10922
94 if bit6(:,z)==1
95     sA(1,2,z)=sA(1,3,z)+1;
96     sA(2,1,z)=sA(1,3,z)-1;
97     sA(2,2,z)=sA(1,3,z)+2;
98     sA(2,3,z)=sA(3,3,z)+1;
99     sA(3,2,z)=sA(3,3,z)-1;
100 elseif bit6(:,z)==0
101     sA(1,2,z)=sA(1,1,z)+1;
102     sA(2,1,z)=sA(1,1,z)-1;
103     sA(2,2,z)=sA(1,1,z)+2;
104     sA(2,3,z)=sA(3,3,z)+1;
105     sA(3,2,z)=sA(3,3,z)-1;
106 end
107 end
108
109 for z=1:10922
110     if sA(1,1,z)==sA(1,3,z)
111         if bit6(:,z)==0
112             sA(1,2,z)=sA(1,1,z)+1;
113             sA(2,1,z)=sA(3,1,z)+1;
114             sA(2,2,z)=sA(3,1,z)+2;
115             sA(2,3,z)=sA(3,3,z)+1;
116             sA(3,2,z)=sA(3,1,z)-1;
117         else if bit6(:,z)==1
118             sA(1,2,z)=sA(1,3,z)+1;
119             sA(2,1,z)=sA(3,1,z)+1;
120             sA(2,2,z)=sA(3,3,z)+2;
121             sA(2,3,z)=sA(3,3,z)+1;
122             sA(3,2,z)=sA(3,3,z)-1;
123
124         end
125     end
126 end
127 end

```

```

128
129 for q=1:16384
130     cover(:, :, q)=sA(:, :, q);
131 end
132
133 cover=uint8(cover);
134 for j=1:10923
135     if temp(:, :, j)==[0 0 0 0 0]
136         sA(1,2,j)=cover(1,2,j);
137         sA(2,1,j)=cover(2,1,j);
138         sA(2,2,j)=cover(2,2,j);
139         sA(2,3,j)=cover(2,3,j);
140         sA(3,2,j)=cover(3,2,j);
141     end
142 end
143 for j=1:10923
144     if temp(:, :, j)==[0 0 0 0 1]
145         sA(1,2,j)=cover(1,2,j);
146         sA(2,1,j)=cover(2,1,j);
147         sA(2,2,j)=cover(2,2,j);
148         sA(2,3,j)=cover(3,2,j);
149         sA(3,2,j)=cover(2,3,j);
150     end
151 end
152
153 for j=1:10923
154     if temp(:, :, j)==[0 0 0 1 0]
155         sA(1,2,j)=cover(2,1,j);
156         sA(2,1,j)=cover(1,2,j);
157         sA(2,2,j)=cover(2,3,j);
158         sA(2,3,j)=cover(2,2,j);
159         sA(3,2,j)=cover(3,2,j);
160     end
161 end
162
163 for j=1:10923
164     if temp(:, :, j)==[0 0 0 1 1]
165         sA(1,2,j)=cover(2,1,j);
166         sA(2,1,j)=cover(1,2,j);
167         sA(2,2,j)=cover(3,2,j);
168         sA(2,3,j)=cover(2,3,j);
169         sA(3,2,j)=cover(2,2,j);
170     end
171 end
172
173 for j=1:10923
174     if temp(:, :, j)==[0 0 1 0 0]
175         sA(1,2,j)=cover(2,1,j);

```

```

176         sA(2,1,j)=cover(2,2,j);
177         sA(2,2,j)=cover(1,2,j);
178         sA(2,3,j)=cover(2,3,j);
179         sA(3,2,j)=cover(3,2,j);
180     end
181 end
182
183 for j=1:10923
184     if temp(:,j)==[0 0 1 0 1]
185         sA(1,2,j)=cover(2,1,j);
186         sA(2,1,j)=cover(2,2,j);
187         sA(2,2,j)=cover(1,2,j);
188         sA(2,3,j)=cover(3,2,j);
189         sA(3,2,j)=cover(2,3,j);
190     end
191 end
192
193 for j=1:10923
194     if temp(:,j)==[0 0 1 1 0]
195         sA(1,2,j)=cover(2,2,j);
196         sA(2,1,j)=cover(2,1,j);
197         sA(2,2,j)=cover(3,2,j);
198         sA(2,3,j)=cover(1,2,j);
199         sA(3,2,j)=cover(2,3,j);
200     end
201 end
202
203 for j=1:10923
204     if temp(:,j)==[0 0 1 1 1]
205         sA(1,2,j)=cover(2,2,j);
206         sA(2,1,j)=cover(2,1,j);
207         sA(2,2,j)=cover(2,3,j);
208         sA(2,3,j)=cover(3,2,j);
209         sA(3,2,j)=cover(1,2,j);
210     end
211 end
212
213 for j=1:10923
214     if temp(:,j)==[0 1 0 0 0]
215         sA(1,2,j)=cover(1,2,j);
216         sA(2,1,j)=cover(3,2,j);
217         sA(2,2,j)=cover(2,1,j);
218         sA(2,3,j)=cover(2,2,j);
219         sA(3,2,j)=cover(2,3,j);
220     end
221 end
222
223 for j=1:10923

```

```

224 if temp(:,j)==[0 1 0 0 1]
225     sA(1,2,j)=cover(1,2,j);
226     sA(2,1,j)=cover(3,2,j);
227     sA(2,2,j)=cover(2,2,j);
228     sA(2,3,j)=cover(2,3,j);
229     sA(3,2,j)=cover(2,1,j);
230 end
231 end
232
233 for j=1:10923
234 if temp(:,j)==[0 1 0 1 0]
235     sA(1,2,j)=cover(3,2,j);
236     sA(2,1,j)=cover(1,2,j);
237     sA(2,2,j)=cover(2,1,j);
238     sA(2,3,j)=cover(2,2,j);
239     sA(3,2,j)=cover(2,3,j);
240 end
241 end
242
243 for j=1:10923
244 if temp(:,j)==[0 1 0 1 1]
245     sA(1,2,j)=cover(3,2,j);
246     sA(2,1,j)=cover(1,2,j);
247     sA(2,2,j)=cover(2,2,j);
248     sA(2,3,j)=cover(2,1,j);
249     sA(3,2,j)=cover(2,3,j);
250 end
251 end
252
253 for j=1:10923
254 if temp(:,j)==[0 1 1 0 0]
255     sA(1,2,j)=cover(3,2,j);
256     sA(2,1,j)=cover(2,2,j);
257     sA(2,2,j)=cover(2,1,j);
258     sA(2,3,j)=cover(2,3,j);
259     sA(3,2,j)=cover(1,2,j);
260 end
261 end
262 for j=1:10923
263 if temp(:,j)==[0 1 1 0 1]
264     sA(1,2,j)=cover(3,2,j);
265     sA(2,1,j)=cover(2,2,j);
266     sA(2,2,j)=cover(2,3,j);
267     sA(2,3,j)=cover(2,1,j);
268     sA(3,2,j)=cover(1,2,j);
269 end
270 end
271

```

```

272 for j=1:10923
273 if temp(:,j)==[0 1 1 1 0]
274     sA(1,2,j)=cover(2,2,j);
275     sA(2,1,j)=cover(3,2,j);
276     sA(2,2,j)=cover(1,2,j);
277     sA(2,3,j)=cover(2,1,j);
278     sA(3,2,j)=cover(2,3,j);
279 end
280 end
281
282 for j=1:10923
283 if temp(:,j)==[0 1 1 1 1]
284     sA(1,2,j)=cover(2,2,j);
285     sA(2,1,j)=cover(3,2,j);
286     sA(2,2,j)=cover(2,1,j);
287     sA(2,3,j)=cover(2,3,j);
288     sA(3,2,j)=cover(1,2,j);
289 end
290 end
291
292 for j=1:10923
293 if temp(:,j)==[1 0 0 0 0]
294     sA(1,2,j)=cover(1,2,j);
295     sA(2,1,j)=cover(2,2,j);
296     sA(2,2,j)=cover(2,3,j);
297     sA(2,3,j)=cover(2,1,j);
298     sA(3,2,j)=cover(3,2,j);
299
300
301 end
302 end
303
304 for j=1:10923
305 if temp(:,j)==[1 0 0 0 1]
306     sA(1,2,j)=cover(1,2,j);
307     sA(2,1,j)=cover(2,2,j);
308     sA(2,2,j)=cover(3,2,j);
309     sA(2,3,j)=cover(2,1,j);
310     sA(3,2,j)=cover(2,3,j);
311 end
312 end
313
314 for j=1:10923
315 if temp(:,j)==[1 0 0 1 0]
316     sA(1,2,j)=cover(2,1,j);
317     sA(2,1,j)=cover(2,3,j);
318     sA(2,2,j)=cover(2,2,j);
319     sA(2,3,j)=cover(1,2,j);

```

```

320         sA(3,2,j)=cover(3,2,j);
321     end
322 end
323
324 for j=1:10923
325     if temp(:,j)==[1 0 0 1 1]
326         sA(1,2,j)=cover(2,1,j);
327         sA(2,1,j)=cover(2,3,j);
328         sA(2,2,j)=cover(1,2,j);
329         sA(2,3,j)=cover(2,2,j);
330         sA(3,2,j)=cover(3,2,j);
331     end
332 end
333
334 for j=1:10923
335     if temp(:,j)==[1 0 1 0 0]
336         sA(1,2,j)=cover(2,1,j);
337         sA(2,1,j)=cover(3,2,j);
338         sA(2,2,j)=cover(1,2,j);
339         sA(2,3,j)=cover(2,3,j);
340         sA(3,2,j)=cover(2,2,j);
341     end
342 end
343
344 for j=1:10923
345     if temp(:,j)==[1 0 1 0 1]
346         sA(1,2,j)=cover(2,1,j);
347         sA(2,1,j)=cover(3,2,j);
348         sA(2,2,j)=cover(2,3,j);
349         sA(2,3,j)=cover(2,2,j);
350         sA(3,2,j)=cover(1,2,j);
351     end
352 end
353
354 for j=1:10923
355     if temp(:,j)==[1 0 1 1 0]
356         sA(1,2,j)=cover(2,2,j);
357         sA(2,1,j)=cover(1,2,j);
358         sA(2,2,j)=cover(2,1,j);
359         sA(2,3,j)=cover(2,3,j);
360         sA(3,2,j)=cover(3,2,j);
361     end
362 end
363
364 for j=1:10923
365     if temp(:,j)==[1 0 1 1 1]
366         sA(1,2,j)=cover(2,2,j);
367         sA(2,1,j)=cover(1,2,j);

```

```

368         sA(2,2,j)=cover(3,2,j);
369         sA(2,3,j)=cover(2,1,j);
370         sA(3,2,j)=cover(2,3,j);
371     end
372 end
373
374 for j=1:10923
375     if temp(:,j)==[1 1 0 0 0]
376         sA(1,2,j)=cover(1,2,j);
377         sA(2,1,j)=cover(2,3,j);
378         sA(2,2,j)=cover(3,2,j);
379         sA(2,3,j)=cover(2,1,j);
380         sA(3,2,j)=cover(2,2,j);
381     end
382 end
383
384 for j=1:10923
385     if temp(:,j)==[1 1 0 0 1]
386         sA(1,2,j)=cover(1,2,j);
387         sA(2,1,j)=cover(2,3,j);
388         sA(2,2,j)=cover(2,2,j);
389         sA(2,3,j)=cover(3,2,j);
390         sA(3,2,j)=cover(2,1,j);
391     end
392 end
393
394 for j=1:10923
395     if temp(:,j)==[1 1 0 1 0]
396         sA(1,2,j)=cover(3,2,j);
397         sA(2,1,j)=cover(2,1,j);
398         sA(2,2,j)=cover(2,3,j);
399         sA(2,3,j)=cover(1,2,j);
400         sA(3,2,j)=cover(2,2,j);
401     end
402 end
403
404 for j=1:10923
405     if temp(:,j)==[1 1 0 1 1]
406         sA(1,2,j)=cover(3,2,j);
407         sA(2,1,j)=cover(2,1,j);
408         sA(2,2,j)=cover(2,2,j);
409         sA(2,3,j)=cover(2,3,j);
410         sA(3,2,j)=cover(1,2,j);
411     end
412 end
413
414 for j=1:10923
415     if temp(:,j)==[1 1 1 0 0]

```

```

416         sA(1,2,j)=cover(3,2,j);
417         sA(2,1,j)=cover(2,3,j);
418         sA(2,2,j)=cover(2,1,j);
419         sA(2,3,j)=cover(2,2,j);
420         sA(3,2,j)=cover(1,2,j);
421     end
422 end
423
424 for j=1:10923
425     if temp(:, :, j)==[1 1 1 0 1]
426         sA(1,2,j)=cover(3,2,j);
427         sA(2,1,j)=cover(2,3,j);
428         sA(2,2,j)=cover(2,2,j);
429         sA(2,3,j)=cover(1,2,j);
430         sA(3,2,j)=cover(2,1,j);
431     end
432 end
433
434 for j=1:10923
435     if temp(:, :, j)==[1 1 1 1 0]
436         sA(1,2,j)=cover(2,2,j);
437         sA(2,1,j)=cover(2,3,j);
438         sA(2,2,j)=cover(3,2,j);
439         sA(2,3,j)=cover(2,1,j);
440         sA(3,2,j)=cover(1,2,j);
441     end
442 end
443
444 for j=1:10923
445     if temp(:, :, j)==[1 1 1 1 1]
446         sA(1,2,j)=cover(2,2,j);
447         sA(2,1,j)=cover(2,3,j);
448         sA(2,2,j)=cover(2,1,j);
449         sA(2,3,j)=cover(3,2,j);
450         sA(3,2,j)=cover(1,2,j);
451     end
452 end
453
454 sA=uint8(sA);
455 h=1;
456 q1=[sA(:, :, h) to sA(:, :, h+ 127)];
457 h=h+128;
458 q2=[sA(:, :, h) to sA(:, :, h+ 127)];
459 h=h+128;
460 q3=[sA(:, :, h) to sA(:, :, h+ 127)];
461 h=h+128;
462 q4=[sA(:, :, h) to sA(:, :, h+ 127)];
463 h=h+128;

```

```

464 q5=[sA(:, :, h) to sA(:, :, h+ 127];
465 h=h+128;
466 q6=[sA(:, :, h) to sA(:, :, h+ 127];
467 h=h+128;
468 q7=[sA(:, :, h) to sA(:, :, h+ 127];
469 h=h+128;
470 q8=[sA(:, :, h) to sA(:, :, h+ 127];
471 h=h+128;
472 q9=[sA(:, :, h) to sA(:, :, h+ 127];
473 h=h+128;
474 q10=[sA(:, :, h) to sA(:, :, h+ 127];
475 h=h+128;
476 q11=[sA(:, :, h) to sA(:, :, h+ 127];
477 h=h+128;
478 q12=[sA(:, :, h) to sA(:, :, h+ 127];
479 h=h+128;
480 q13=[sA(:, :, h) to sA(:, :, h+ 127];
481 h=h+128;
482 q14=[sA(:, :, h) to sA(:, :, h+ 127];
483 h=h+128;
484
485
486
487
488
489
490
491
492
493
494
495 upto
496 q128=[sA(:, :, h) to sA(:, :, h+ 127];
497
498
499
500 stegomatrix=[q1;q2;q3;q4;q5;q6;q7;q8;q9;q10;q11;q12;q13;q14;q15;q16;q17;q18;q19;q20;
               q21;q22;q23;q24;q25; ..... q120;q121;q122;q123;q124;q125;q126;q127;
               q128];
501
502 figure ,imshow(stegomatrix)
503 imsave()

```

4.2 Data Extraction

```
1  simg=imread("C:\Users\91863\Documents\MATLAB\lenastego.bmp");
2  figure ,imshow(simg)
3
4  [r , c]=size (simg);
5
6  rs=3;
7
8  cs=3;
9
10 nr=floor(r/rs);
11
12 nc=floor(r/cs);
13
14 simg1(rs , cs , nr*nc)=0;
15 z=1;
16
17 for i=1:nr
18     for j=1:nc
19         simg1 (: , : , z) = simg(((i-1)*rs+1):rs*i , ((j-1)*cs+1):cs*j);
20
21         z=z+1;
22
23     end
24 end
25 simg0=simg1;
26 for z=1:16384
27     si(1,1,z)=simg0(1,1,z);
28     si(1,2,z)=simg0(1,3,z);
29     si(2,1,z)=simg0(3,1,z);
30     si(2,2,z)=simg0(3,3,z);
31 end
32
33 sA=si;
34
35 h=1;
36
37 q1=[sA(:, :, h) to sA(:, :, h+ 127)];
38 h=h+128;
39 q2=[sA(:, :, h) to sA(:, :, h+ 127)];
40 h=h+128;
41 q3=[sA(:, :, h) to sA(:, :, h+ 127)];
42 h=h+128;
43 q4=[sA(:, :, h) to sA(:, :, h+ 127)];
44 h=h+128;
45 q5=[sA(:, :, h) to sA(:, :, h+ 127)];
46 h=h+128;
```

```

47 q6=[sA(:, :, h) to sA(:, :, h+ 127];
48 h=h+128;
49 q7=[sA(:, :, h) to sA(:, :, h+ 127];
50 h=h+128;
51 q8=[sA(:, :, h) to sA(:, :, h+ 127];
52 h=h+128;
53 q9=[sA(:, :, h) to sA(:, :, h+ 127];
54 h=h+128;
55 q10=[sA(:, :, h) to sA(:, :, h+ 127];
56 h=h+128;
57 q11=[sA(:, :, h) to sA(:, :, h+ 127];
58 h=h+128;
59 q12=[sA(:, :, h) to sA(:, :, h+ 127];
60 h=h+128;
61 q13=[sA(:, :, h) to sA(:, :, h+ 127];
62 h=h+128;
63 q14=[sA(:, :, h) to sA(:, :, h+ 127];
64 h=h+128;
65
66
67
68
69
70
71
72
73
74
75
76 upto
77 q128=[sA(:, :, h) to sA(:, :, h+ 127];
78
79
80 covermatrix=[q1;q2;q3;q4;q5;q6;q7;q8;q9;q10;q11;q12;q13;q14;q15;q16;q17;q18;q19;q20
;.....; q121;q122;q123;q124;q125;q126;q127;q128];
81 covermatrix=uint8(covermatrix);
82 figure, imshow(covermatrix)
83
84 for z=1:16384
85     if simg1(1,1,z)~=simg1(1,3,z)
86         if (simg1(1,2,z)==(simg1(1,1,z)+1)&&simg1(2,1,z)==(simg1(1,1,z)-1)&&simg1(2,2,z)
==(simg1(1,1,z)+2)&&simg1(2,3,z)==(simg1(3,3,z)+1)&&simg1(3,2,z)==(simg1(3,3,z)-1)
)
87             spixel(:, :, z)= [0 0 0 0 0 0];
88         else if (simg1(1,2,z)==(simg1(1,3,z)+1)&&simg1(2,1,z)==(simg1(1,3,z)-1)&&simg1
(2,2,z)==(simg1(1,3,z)+2)&&simg1(2,3,z)==(simg1(3,3,z)+1)&&simg1(3,2,z)==(simg1
(3,3,z)-1))
89             spixel(:, :, z)= [0 0 0 0 0 1];

```

```

90     else if (simgl(1,2,z)==(simgl(1,1,z)+1)&&simgl(2,1,z)==(simgl(1,1,z)-1)&&simgl
(2,2,z)==(simgl(1,1,z)+2)&&simgl(2,3,z)==(simgl(3,3,z)-1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
91         spixel(:, :, z) = [0 0 0 0 1 0];
92     else if (simgl(1,2,z)==(simgl(1,3,z)+1)&&simgl(2,1,z)==(simgl(1,3,z)-1)&&simgl
(2,2,z)==(simgl(1,3,z)+2)&&simgl(2,3,z)==(simgl(3,3,z)-1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
93         spixel(:, :, z) = [0 0 0 0 1 1];
94     else if (simgl(1,2,z)==(simgl(1,1,z)-1)&&simgl(2,1,z)==(simgl(1,1,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(1,1,z)+2)&&simgl(3,2,z)==(simgl
(3,3,z)-1))
95         spixel(:, :, z) = [0 0 0 1 0 0];
96     else if (simgl(1,2,z)==(simgl(1,3,z)-1)&&simgl(2,1,z)==(simgl(1,3,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(1,3,z)+2)&&simgl(3,2,z)==(simgl
(3,3,z)-1))
97         spixel(:, :, z) = [0 0 0 1 0 1];
98     else if (simgl(1,2,z)==(simgl(1,1,z)-1)&&simgl(2,1,z)==(simgl(1,1,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)-1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,1,z)+2))
99         spixel(:, :, z) = [0 0 0 1 1 0];
100    else if (simgl(1,2,z)==(simgl(1,3,z)-1)&&simgl(2,1,z)==(simgl(1,3,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)-1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,3,z)+2))
101        spixel(:, :, z) = [0 0 0 1 1 1];
102    else if (simgl(1,2,z)==(simgl(1,1,z)-1)&&simgl(2,1,z)==(simgl(1,1,z)+2)&&simgl
(2,2,z)==(simgl(1,1,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)-1))
103        spixel(:, :, z) = [0 0 1 0 0 0];
104    else if (simgl(1,2,z)==(simgl(1,3,z)-1)&&simgl(2,1,z)==(simgl(1,3,z)+2)&&simgl
(2,2,z)==(simgl(1,3,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)-1))
105        spixel(:, :, z) = [0 0 1 0 0 1];
106    else if (simgl(1,2,z)==(simgl(1,1,z)-1)&&simgl(2,1,z)==(simgl(1,1,z)+2)&&
simgl(2,2,z)==(simgl(1,1,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)-1)&&simgl(3,2,z)==(
simgl(3,3,z)+1))
107        spixel(:, :, z) = [0 0 1 0 1 0];
108    else if (simgl(1,2,z)==(simgl(1,3,z)-1)&&simgl(2,1,z)==(simgl(1,3,z)+2)&&
simgl(2,2,z)==(simgl(1,3,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)-1)&&simgl(3,2,z)==(
simgl(3,3,z)+1))
109        spixel(:, :, z) = [0 0 1 0 1 1];
110    else if (simgl(1,2,z)==(simgl(1,1,z)+2)&&simgl(2,1,z)==(simgl(1,1,z)-1)&&
simgl(2,2,z)==(simgl(3,3,z)-1)&&simgl(2,3,z)==(simgl(1,1,z)+1)&&simgl(3,2,z)==(
simgl(3,3,z)+1))
111        spixel(:, :, z) = [0 0 1 1 0 0];
112    else if (simgl(1,2,z)==(simgl(1,3,z)+2)&&simgl(2,1,z)==(simgl(1,3,z)-1)&&simgl
(2,2,z)==(simgl(3,3,z)-1)&&simgl(2,3,z)==(simgl(1,3,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
113        spixel(:, :, z) = [0 0 1 1 0 1];

```

```

114         else if (simgl(1,2,z)==(simgl(1,1,z)+2)&&simgl(2,1,z)==(simgl(1,1,z)-1)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)-1)&&simgl(3,2,z)==(simgl
(1,1,z)+1))
115             spixel(:, :, z) = [0 0 1 1 1 0];
116         else if (simgl(1,2,z)==(simgl(1,3,z)+2)&&simgl(2,1,z)==(simgl(1,3,z)-1)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)-1)&&simgl(3,2,z)==(simgl
(1,3,z)+1))
117             spixel(:, :, z) = [0 0 1 1 1 1];
118         else if (simgl(1,2,z)==(simgl(1,1,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(1,1,z)-1)&&simgl(2,3,z)==(simgl(1,1,z)+2)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
119             spixel(:, :, z) = [0 1 0 0 0 0];
120         else if (simgl(1,2,z)==(simgl(1,3,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(1,3,z)-1)&&simgl(2,3,z)==(simgl(1,3,z)+2)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
121             spixel(:, :, z) = [0 1 0 0 0 1];
122         else if (simgl(1,2,z)==(simgl(1,1,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(1,1,z)+2)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,1,z)-1))
123             spixel(:, :, z) = [0 1 0 0 1 0];
124         else if (simgl(1,2,z)==(simgl(1,3,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(1,3,z)+2)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,3,z)-1))
125             spixel(:, :, z) = [0 1 0 0 1 1];
126         else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(1,1,z)+1)&&simgl
(2,2,z)==(simgl(1,1,z)-1)&&simgl(2,3,z)==(simgl(1,1,z)+2)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
127             spixel(:, :, z) = [0 1 0 1 0 0];
128         else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(1,3,z)+1)&&simgl
(2,2,z)==(simgl(1,3,z)-1)&&simgl(2,3,z)==(simgl(1,3,z)+2)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
129             spixel(:, :, z) = [0 1 0 1 0 1];
130         else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(1,1,z)+1)&&simgl
(2,2,z)==(simgl(1,1,z)+2)&&simgl(2,3,z)==(simgl(1,1,z)-1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
131             spixel(:, :, z) = [0 1 0 1 1 0];
132         else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(1,3,z)+1)&&simgl
(2,2,z)==(simgl(1,3,z)+2)&&simgl(2,3,z)==(simgl(1,3,z)-1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
133             spixel(:, :, z) = [0 1 0 1 1 1];
134         else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(1,1,z)+2)&&simgl
(2,2,z)==(simgl(1,1,z)-1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,1,z)+1))
135             spixel(:, :, z) = [0 1 1 0 0 0];
136         else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(1,3,z)+2)&&simgl
(2,2,z)==(simgl(1,3,z)-1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,3,z)+1))
137             spixel(:, :, z) = [0 1 1 0 0 1];

```

```

138     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(1,1,z)+2)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(1,1,z)-1)&&simgl(3,2,z)==(simgl
(1,1,z)+1))
139         spixel(:, :, z) = [0 1 1 0 1 0];
140     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(1,3,z)+2)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(1,3,z)-1)&&simgl(3,2,z)==(simgl
(1,3,z)+1))
141         spixel(:, :, z) = [0 1 1 0 1 1];
142     else if (simgl(1,2,z)==(simgl(1,1,z)+2)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(1,1,z)+1)&&simgl(2,3,z)==(simgl(1,1,z)-1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
143         spixel(:, :, z) = [0 1 1 1 0 0];
144     else if (simgl(1,2,z)==(simgl(1,3,z)+2)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(1,3,z)+1)&&simgl(2,3,z)==(simgl(1,3,z)-1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
145         spixel(:, :, z) = [0 1 1 1 0 1];
146     else if (simgl(1,2,z)==(simgl(1,1,z)+2)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(1,1,z)-1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,1,z)+1))
147         spixel(:, :, z) = [0 1 1 1 1 0];
148     else if (simgl(1,2,z)==(simgl(1,3,z)+2)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(1,3,z)-1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,3,z)+1))
149         spixel(:, :, z) = [0 1 1 1 1 1];
150     else if (simgl(1,2,z)==(simgl(1,1,z)+1)&&simgl(2,1,z)==(simgl(1,1,z)+2)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(1,1,z)-1)&&simgl(3,2,z)==(simgl
(3,3,z)-1))
151         spixel(:, :, z) = [1 0 0 0 0 0];
152     else if (simgl(1,2,z)==(simgl(1,3,z)+1)&&simgl(2,1,z)==(simgl(1,3,z)+2)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(1,3,z)-1)&&simgl(3,2,z)==(simgl
(3,3,z)-1))
153         spixel(:, :, z) = [1 0 0 0 0 1];
154     else if (simgl(1,2,z)==(simgl(1,1,z)+1)&&simgl(2,1,z)==(simgl(1,1,z)+2)&&simgl
(2,2,z)==(simgl(3,3,z)-1)&&simgl(2,3,z)==(simgl(1,1,z)-1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
155         spixel(:, :, z) = [1 0 0 0 1 0];
156     else if (simgl(1,2,z)==(simgl(1,3,z)+1)&&simgl(2,1,z)==(simgl(1,3,z)+2)&&simgl
(2,2,z)==(simgl(3,3,z)-1)&&simgl(2,3,z)==(simgl(1,3,z)-1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
157         spixel(:, :, z) = [1 0 0 0 1 1];
158     else if (simgl(1,2,z)==(simgl(1,1,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(1,1,z)+2)&&simgl(2,3,z)==(simgl(1,1,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)-1))
159         spixel(:, :, z) = [1 0 0 1 0 0];
160     else if (simgl(1,2,z)==(simgl(1,3,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(1,3,z)+2)&&simgl(2,3,z)==(simgl(1,3,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)-1))
161         spixel(:, :, z) = [1 0 0 1 0 1];

```

```

162     else if (simgl(1,2,z)==(simgl(1,1,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(1,1,z)+1)&&simgl(2,3,z)==(simgl(1,1,z)+2)&&simgl(3,2,z)==(simgl
(3,3,z)-1))
163         spixel(:, :, z) = [1 0 0 1 1 0];
164     else if (simgl(1,2,z)==(simgl(1,3,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(1,3,z)+1)&&simgl(2,3,z)==(simgl(1,3,z)+2)&&simgl(3,2,z)==(simgl
(3,3,z)-1))
165         spixel(:, :, z) = [1 0 0 1 1 1];
166     else if (simgl(1,2,z)==(simgl(1,1,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(1,1,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,1,z)+2))
167         spixel(:, :, z) = [1 0 1 0 0 0];
168     else if (simgl(1,2,z)==(simgl(1,3,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(1,3,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,3,z)+2))
169         spixel(:, :, z) = [1 0 1 0 0 1];
170     else if (simgl(1,2,z)==(simgl(1,1,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(1,1,z)+2)&&simgl(3,2,z)==(simgl
(1,1,z)+1))
171         spixel(:, :, z) = [1 0 1 0 1 0];
172     else if (simgl(1,2,z)==(simgl(1,3,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(1,3,z)+2)&&simgl(3,2,z)==(simgl
(1,3,z)+1))
173         spixel(:, :, z) = [1 0 1 0 1 1];
174     else if (simgl(1,2,z)==(simgl(1,1,z)+2)&&simgl(2,1,z)==(simgl(1,1,z)+1)&&simgl
(2,2,z)==(simgl(1,1,z)-1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)-1))
175         spixel(:, :, z) = [1 0 1 1 0 0];
176     else if (simgl(1,2,z)==(simgl(1,3,z)+2)&&simgl(2,1,z)==(simgl(1,3,z)+1)&&simgl
(2,2,z)==(simgl(1,3,z)-1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)-1))
177         spixel(:, :, z) = [1 0 1 1 0 1];
178     else if (simgl(1,2,z)==(simgl(1,1,z)+2)&&simgl(2,1,z)==(simgl(1,1,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)-1)&&simgl(2,3,z)==(simgl(1,1,z)-1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
179         spixel(:, :, z) = [1 0 1 1 1 0];
180     else if (simgl(1,2,z)==(simgl(1,3,z)+2)&&simgl(2,1,z)==(simgl(1,3,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)-1)&&simgl(2,3,z)==(simgl(1,3,z)-1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
181         spixel(:, :, z) = [1 0 1 1 1 1];
182     else if (simgl(1,2,z)==(simgl(1,1,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)-1)&&simgl(2,3,z)==(simgl(1,1,z)-1)&&simgl(3,2,z)==(simgl
(1,1,z)+2))
183         spixel(:, :, z) = [1 1 0 0 0 0];
184     else if (simgl(1,2,z)==(simgl(1,3,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)-1)&&simgl(2,3,z)==(simgl(1,3,z)-1)&&simgl(3,2,z)==(simgl
(1,3,z)+2))
185         spixel(:, :, z) = [1 1 0 0 0 1];

```

```

186     else if (simgl(1,2,z)==(simgl(1,1,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(1,1,z)+2)&&simgl(2,3,z)==(simgl(3,3,z)-1)&&simgl(3,2,z)==(simgl
(1,1,z)-1))
187         spixel(:, :, z) = [1 1 0 0 1 0];
188     else if (simgl(1,2,z)==(simgl(1,3,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(1,3,z)+2)&&simgl(2,3,z)==(simgl(3,3,z)-1)&&simgl(3,2,z)==(simgl
(1,3,z)-1))
189         spixel(:, :, z) = [1 1 0 0 1 1];
190     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(1,1,z)-1)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(1,1,z)+1)&&simgl(3,2,z)==(simgl
(1,1,z)+2))
191         spixel(:, :, z) = [1 1 0 1 0 0];
192     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(1,3,z)-1)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(1,3,z)+1)&&simgl(3,2,z)==(simgl
(1,3,z)+2))
193         spixel(:, :, z) = [1 1 0 1 0 1];
194     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(1,1,z)-1)&&simgl
(2,2,z)==(simgl(1,1,z)+2)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,1,z)+1))
195         spixel(:, :, z) = [1 1 0 1 1 0];
196     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(1,3,z)-1)&&simgl
(2,2,z)==(simgl(1,3,z)+2)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,3,z)+1))
197         spixel(:, :, z) = [1 1 0 1 1 1];
198     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(1,1,z)-1)&&simgl(2,3,z)==(simgl(1,1,z)+2)&&simgl(3,2,z)==(simgl
(1,1,z)+1))
199         spixel(:, :, z) = [1 1 1 0 0 0];
200     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(1,3,z)-1)&&simgl(2,3,z)==(simgl(1,3,z)+2)&&simgl(3,2,z)==(simgl
(1,3,z)+1))
201         spixel(:, :, z) = [1 1 1 0 0 1];
202     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(1,1,z)+2)&&simgl(2,3,z)==(simgl(1,1,z)+1)&&simgl(3,2,z)==(simgl
(1,1,z)-1))
203         spixel(:, :, z) = [1 1 1 0 1 0];
204     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(1,3,z)+2)&&simgl(2,3,z)==(simgl(1,3,z)+1)&&simgl(3,2,z)==(simgl
(1,3,z)-1))
205         spixel(:, :, z) = [1 1 1 0 1 1];
206     else if (simgl(1,2,z)==(simgl(1,1,z)+2)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)-1)&&simgl(2,3,z)==(simgl(1,1,z)-1)&&simgl(3,2,z)==(simgl
(1,1,z)+1))
207         spixel(:, :, z) = [1 1 1 1 0 0];
208     else if (simgl(1,2,z)==(simgl(1,3,z)+2)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)-1)&&simgl(2,3,z)==(simgl(1,3,z)-1)&&simgl(3,2,z)==(simgl
(1,3,z)+1))
209         spixel(:, :, z) = [1 1 1 1 0 1];

```

```

210     else if (sing1(1,2,z)==(sing1(1,1,z)+2)&&sing1(2,1,z)==(sing1(3,3,z)+1)&&sing1
(2,2,z)==(sing1(1,1,z)-1)&&sing1(2,3,z)==(sing1(3,3,z)-1)&&sing1(3,2,z)==(sing1
(1,1,z)+1))
211         spixel(:,z)= [1 1 1 1 1 0];
212     else if (sing1(1,2,z)==(sing1(1,3,z)+2)&&sing1(2,1,z)==(sing1(3,3,z)+1)&&sing1
(2,2,z)==(sing1(1,3,z)-1)&&sing1(2,3,z)==(sing1(3,3,z)-1)&&sing1(3,2,z)==(sing1
(1,3,z)+1))
213         spixel(:,z)= [1 1 1 1 1 1];
214     end
215     end
216     end
217     end
218     end
219     end
220     end
221     end
222     end
223     end
224     end
225     end
226     end
227     end
228     end
229     end
230     end
231     end
232     end
233     end
234     end
235     end
236     end
237     end
238     end
239     end
240     end
241     end
242     end
243     end
244     end
245     end
246     end
247     end
248     end
249     end
250     end
251     end
252     end
253     end

```

```

254         end
255     end
256 end
257 end
258 end
259 end
260 end
261 end
262 end
263 end
264 end
265 end
266     end
267 end
268 end
269 end
270 end
271 end
272 end
273 end
274 end
275 end
276 end
277 end
278 end
279 end
280
281 for z=1:16384
282     if sing1(1,1,z)==sing1(1,3,z)
283         if (sing1(1,2,z)==(sing1(1,1,z)+1)&&sing1(2,1,z)==(sing1(3,1,z)+1)&&sing1(2,2,z)==(
            sing1(3,1,z)+2)&&sing1(2,3,z)==(sing1(3,3,z)+1)&&sing1(3,2,z)==(sing1(3,1,z)-1))
284             spixel(:, :, z)= [0 0 0 0 0 0];
285         else if (sing1(1,2,z)==(sing1(1,3,z)+1)&&sing1(2,1,z)==(sing1(3,1,z)+1)&&sing1
            (2,2,z)==(sing1(3,3,z)+2)&&sing1(2,3,z)==(sing1(3,3,z)+1)&&sing1(3,2,z)==(sing1
            (3,3,z)-1))
286             spixel(:, :, z)= [0 0 0 0 0 1];
287         else if (sing1(1,2,z)==(sing1(1,1,z)+1)&&sing1(2,1,z)==(sing1(3,1,z)+1)&&sing1
            (2,2,z)==(sing1(3,1,z)+2)&&sing1(2,3,z)==(sing1(3,1,z)-1)&&sing1(3,2,z)==(sing1
            (3,3,z)+1))
288             spixel(:, :, z)= [0 0 0 0 1 0];
289         else if (sing1(1,2,z)==(sing1(1,3,z)+1)&&sing1(2,1,z)==(sing1(3,1,z)+1)&&sing1
            (2,2,z)==(sing1(3,3,z)+2)&&sing1(2,3,z)==(sing1(3,3,z)-1)&&sing1(3,2,z)==(sing1
            (3,3,z)+1))
290             spixel(:, :, z)= [0 0 0 0 1 1];
291         else if (sing1(1,2,z)==(sing1(3,1,z)+1)&&sing1(2,1,z)==(sing1(1,1,z)+1)&&sing1
            (2,2,z)==(sing1(3,3,z)+1)&&sing1(2,3,z)==(sing1(3,1,z)+2)&&sing1(3,2,z)==(sing1
            (3,1,z)-1))
292             spixel(:, :, z)= [0 0 0 1 0 0];

```

```

293     else if (simg1(1,2,z)==(simg1(3,1,z)+1)&&simg1(2,1,z)==(simg1(1,3,z)+1)&&simg1
(2,2,z)==(simg1(3,3,z)+1)&&simg1(2,3,z)==(simg1(3,3,z)+2)&&simg1(3,2,z)==(simg1
(3,3,z)-1))
294         spixel(:,z)= [0 0 0 1 0 1];
295     else if (simg1(1,2,z)==(simg1(3,1,z)+1)&&simg1(2,1,z)==(simg1(1,1,z)+1)&&simg1
(2,2,z)==(simg1(3,1,z)-1)&&simg1(2,3,z)==(simg1(3,3,z)+1)&&simg1(3,2,z)==(simg1
(3,1,z)+2))
296         spixel(:,z)= [0 0 0 1 1 0];
297     else if (simg1(1,2,z)==(simg1(3,1,z)+1)&&simg1(2,1,z)==(simg1(1,3,z)+1)&&simg1
(2,2,z)==(simg1(3,3,z)-1)&&simg1(2,3,z)==(simg1(3,3,z)+1)&&simg1(3,2,z)==(simg1
(3,3,z)+2))
298         spixel(:,z)= [0 0 0 1 1 1];
299     else if (simg1(1,2,z)==(simg1(3,1,z)+1)&&simg1(2,1,z)==(simg1(3,1,z)+2)&&simg1
(2,2,z)==(simg1(1,1,z)+1)&&simg1(2,3,z)==(simg1(3,3,z)+1)&&simg1(3,2,z)==(simg1
(3,1,z)-1))
300         spixel(:,z)= [0 0 1 0 0 0];
301     else if (simg1(1,2,z)==(simg1(3,1,z)+1)&&simg1(2,1,z)==(simg1(3,3,z)+2)&&simg1
(2,2,z)==(simg1(1,3,z)+1)&&simg1(2,3,z)==(simg1(3,3,z)+1)&&simg1(3,2,z)==(simg1
(3,3,z)-1))
302         spixel(:,z)= [0 0 1 0 0 1];
303     else if (simg1(1,2,z)==(simg1(3,1,z)+1)&&simg1(2,1,z)==(simg1(3,1,z)+2)&&
simg1(2,2,z)==(simg1(1,1,z)+1)&&simg1(2,3,z)==(simg1(3,1,z)-1)&&simg1(3,2,z)==(
simg1(3,3,z)+1))
304         spixel(:,z)= [0 0 1 0 1 0];
305     else if (simg1(1,2,z)==(simg1(3,1,z)-1)&&simg1(2,1,z)==(simg1(3,3,z)+2)&&simg1
(2,2,z)==(simg1(1,3,z)+1)&&simg1(2,3,z)==(simg1(3,3,z)-1)&&simg1(3,2,z)==(simg1
(3,3,z)+1))
306         spixel(:,z)= [0 0 1 0 1 1];
307     else if (simg1(1,2,z)==(simg1(3,1,z)+2)&&simg1(2,1,z)==(simg1(3,1,z)+1)&&
simg1(2,2,z)==(simg1(3,1,z)-1)&&simg1(2,3,z)==(simg1(1,1,z)+1)&&simg1(3,2,z)==(
simg1(3,3,z)+1))
308         spixel(:,z)= [0 0 1 1 0 0];
309     else if (simg1(1,2,z)==(simg1(3,3,z)+2)&&simg1(2,1,z)==(simg1(3,1,z)+1)&&simg1
(2,2,z)==(simg1(3,3,z)-1)&&simg1(2,3,z)==(simg1(1,3,z)+1)&&simg1(3,2,z)==(simg1
(3,3,z)+1))
310         spixel(:,z)= [0 0 1 1 0 1];
311     else if (simg1(1,2,z)==(simg1(3,1,z)+2)&&simg1(2,1,z)==(simg1(3,1,z)+1)&&simg1
(2,2,z)==(simg1(3,3,z)+1)&&simg1(2,3,z)==(simg1(3,1,z)-1)&&simg1(3,2,z)==(simg1
(1,1,z)+1))
312         spixel(:,z)= [0 0 1 1 1 0];
313     else if (simg1(1,2,z)==(simg1(3,3,z)+2)&&simg1(2,1,z)==(simg1(3,1,z)+1)&&simg1
(2,2,z)==(simg1(3,3,z)+1)&&simg1(2,3,z)==(simg1(3,3,z)-1)&&simg1(3,2,z)==(simg1
(1,3,z)+1))
314         spixel(:,z)= [0 0 1 1 1 1];
315     else if (simg1(1,2,z)==(simg1(1,1,z)+1)&&simg1(2,1,z)==(simg1(3,1,z)-1)&&simg1
(2,2,z)==(simg1(3,1,z)+1)&&simg1(2,3,z)==(simg1(3,1,z)+2)&&simg1(3,2,z)==(simg1
(3,3,z)+1))
316         spixel(:,z)= [0 1 0 0 0 0];

```

```

317     else if (simgl(1,2,z)==(simgl(1,3,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(3,1,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)+2)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
318         spixel(:, :, z) = [0 1 0 0 0 1];
319     else if (simgl(1,2,z)==(simgl(1,1,z)+1)&&simgl(2,1,z)==(simgl(3,1,z)-1)&&simgl
(2,2,z)==(simgl(3,1,z)+2)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(3,1,z)+1))
320         spixel(:, :, z) = [0 1 0 0 1 0];
321     else if (simgl(1,2,z)==(simgl(1,3,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(3,3,z)+2)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(3,1,z)+1))
322         spixel(:, :, z) = [0 1 0 0 1 1];
323     else if (simgl(1,2,z)==(simgl(3,1,z)-1)&&simgl(2,1,z)==(simgl(1,1,z)+1)&&simgl
(2,2,z)==(simgl(3,1,z)+1)&&simgl(2,3,z)==(simgl(3,1,z)+2)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
324         spixel(:, :, z) = [0 1 0 1 0 0];
325     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(1,3,z)+1)&&simgl
(2,2,z)==(simgl(3,1,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)+2)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
326         spixel(:, :, z) = [0 1 0 1 0 1];
327     else if (simgl(1,2,z)==(simgl(3,1,z)-1)&&simgl(2,1,z)==(simgl(1,1,z)+1)&&simgl
(2,2,z)==(simgl(3,1,z)+2)&&simgl(2,3,z)==(simgl(3,1,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
328         spixel(:, :, z) = [0 1 0 1 1 0];
329     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(1,3,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)+2)&&simgl(2,3,z)==(simgl(3,1,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
330         spixel(:, :, z) = [0 1 0 1 1 1];
331     else if (simgl(1,2,z)==(simgl(3,1,z)-1)&&simgl(2,1,z)==(simgl(3,1,z)+2)&&simgl
(2,2,z)==(simgl(3,1,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,1,z)+1))
332         spixel(:, :, z) = [0 1 1 0 0 0];
333     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)+2)&&simgl
(2,2,z)==(simgl(3,1,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,3,z)+1))
334         spixel(:, :, z) = [0 1 1 0 0 1];
335     else if (simgl(1,2,z)==(simgl(3,1,z)-1)&&simgl(2,1,z)==(simgl(3,1,z)+2)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(3,1,z)+1)&&simgl(3,2,z)==(simgl
(1,1,z)+1))
336         spixel(:, :, z) = [0 1 1 0 1 0];
337     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)+2)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(3,1,z)+1)&&simgl(3,2,z)==(simgl
(1,3,z)+1))
338         spixel(:, :, z) = [0 1 1 0 1 1];
339     else if (simgl(1,2,z)==(simgl(3,1,z)+2)&&simgl(2,1,z)==(simgl(3,1,z)-1)&&simgl
(2,2,z)==(simgl(1,1,z)+1)&&simgl(2,3,z)==(simgl(3,1,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
340         spixel(:, :, z) = [0 1 1 1 0 0];

```

```

341     else if (simgl(1,2,z)==(simgl(3,3,z)+2)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(1,3,z)+1)&&simgl(2,3,z)==(simgl(3,1,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
342         spixel(:, :, z) = [0 1 1 1 0 1];
343     else if (simgl(1,2,z)==(simgl(3,1,z)+2)&&simgl(2,1,z)==(simgl(3,1,z)-1)&&simgl
(2,2,z)==(simgl(3,1,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,1,z)+1))
344         spixel(:, :, z) = [0 1 1 1 1 0];
345     else if (simgl(1,2,z)==(simgl(3,3,z)+2)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(3,1,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,3,z)+1))
346         spixel(:, :, z) = [0 1 1 1 1 1];
347     else if (simgl(1,2,z)==(simgl(1,1,z)+1)&&simgl(2,1,z)==(simgl(3,1,z)+2)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(3,1,z)+1)&&simgl(3,2,z)==(simgl
(3,1,z)-1))
348         spixel(:, :, z) = [1 0 0 0 0 0];
349     else if (simgl(1,2,z)==(simgl(1,3,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)+2)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(3,1,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)-1))
350         spixel(:, :, z) = [1 0 0 0 0 1];
351     else if (simgl(1,2,z)==(simgl(1,1,z)+1)&&simgl(2,1,z)==(simgl(3,1,z)+2)&&simgl
(2,2,z)==(simgl(3,1,z)-1)&&simgl(2,3,z)==(simgl(3,1,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
352         spixel(:, :, z) = [1 0 0 0 1 0];
353     else if (simgl(1,2,z)==(simgl(1,3,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)+2)&&simgl
(2,2,z)==(simgl(3,3,z)-1)&&simgl(2,3,z)==(simgl(3,1,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
354         spixel(:, :, z) = [1 0 0 0 1 1];
355     else if (simgl(1,2,z)==(simgl(3,1,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,1,z)+2)&&simgl(2,3,z)==(simgl(1,1,z)+1)&&simgl(3,2,z)==(simgl
(3,1,z)-1))
356         spixel(:, :, z) = [1 0 0 1 0 0];
357     else if (simgl(1,2,z)==(simgl(3,1,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)+2)&&simgl(2,3,z)==(simgl(1,3,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)-1))
358         spixel(:, :, z) = [1 0 0 1 0 1];
359     else if (simgl(1,2,z)==(simgl(3,1,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(1,1,z)+1)&&simgl(2,3,z)==(simgl(3,1,z)+2)&&simgl(3,2,z)==(simgl
(3,1,z)-1))
360         spixel(:, :, z) = [1 0 0 1 1 0];
361     else if (simgl(1,2,z)==(simgl(3,1,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(1,3,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)+2)&&simgl(3,2,z)==(simgl
(3,3,z)-1))
362         spixel(:, :, z) = [1 0 0 1 1 1];
363     else if (simgl(1,2,z)==(simgl(3,1,z)+1)&&simgl(2,1,z)==(simgl(3,1,z)-1)&&simgl
(2,2,z)==(simgl(1,1,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(3,1,z)+2))
364         spixel(:, :, z) = [1 0 1 0 0 0];

```

```

365     else if (simgl(1,2,z)==(simgl(3,1,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(1,3,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)+2))
366         spixel(:, :, z) = [1 0 1 0 0 1];
367     else if (simgl(1,2,z)==(simgl(3,1,z)+1)&&simgl(2,1,z)==(simgl(3,1,z)-1)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(3,1,z)+2)&&simgl(3,2,z)==(simgl
(1,1,z)+1))
368         spixel(:, :, z) = [1 0 1 0 1 0];
369     else if (simgl(1,2,z)==(simgl(3,1,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)-1)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)+2)&&simgl(3,2,z)==(simgl
(1,3,z)+1))
370         spixel(:, :, z) = [1 0 1 0 1 1];
371     else if (simgl(1,2,z)==(simgl(3,1,z)+2)&&simgl(2,1,z)==(simgl(1,1,z)+1)&&simgl
(2,2,z)==(simgl(3,1,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(3,1,z)-1))
372         spixel(:, :, z) = [1 0 1 1 0 0];
373     else if (simgl(1,2,z)==(simgl(3,3,z)+2)&&simgl(2,1,z)==(simgl(1,3,z)+1)&&simgl
(2,2,z)==(simgl(3,1,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)-1))
374         spixel(:, :, z) = [1 0 1 1 0 1];
375     else if (simgl(1,2,z)==(simgl(3,1,z)+2)&&simgl(2,1,z)==(simgl(1,1,z)+1)&&simgl
(2,2,z)==(simgl(3,1,z)-1)&&simgl(2,3,z)==(simgl(3,1,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
376         spixel(:, :, z) = [1 0 1 1 1 0];
377     else if (simgl(1,2,z)==(simgl(3,3,z)+2)&&simgl(2,1,z)==(simgl(1,3,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)-1)&&simgl(2,3,z)==(simgl(3,2,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)+1))
378         spixel(:, :, z) = [1 0 1 1 1 1];
379     else if (simgl(1,2,z)==(simgl(1,1,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,1,z)-1)&&simgl(2,3,z)==(simgl(3,1,z)+1)&&simgl(3,2,z)==(simgl
(3,1,z)+2))
380         spixel(:, :, z) = [1 1 0 0 0 0];
381     else if (simgl(1,2,z)==(simgl(1,3,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)-1)&&simgl(2,3,z)==(simgl(3,1,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)+2))
382         spixel(:, :, z) = [1 1 0 0 0 1];
383     else if (simgl(1,2,z)==(simgl(1,1,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,1,z)+2)&&simgl(2,3,z)==(simgl(3,1,z)-1)&&simgl(3,2,z)==(simgl
(3,1,z)+1))
384         spixel(:, :, z) = [1 1 0 0 1 0];
385     else if (simgl(1,2,z)==(simgl(1,3,z)+1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)+2)&&simgl(2,3,z)==(simgl(3,3,z)-1)&&simgl(3,2,z)==(simgl
(3,1,z)+1))
386         spixel(:, :, z) = [1 1 0 0 1 1];
387     else if (simgl(1,2,z)==(simgl(3,1,z)-1)&&simgl(2,1,z)==(simgl(3,1,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(1,1,z)+1)&&simgl(3,2,z)==(simgl
(3,1,z)+2))
388         spixel(:, :, z) = [1 1 0 1 0 0];

```

```

389     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(3,1,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)+1)&&simgl(2,3,z)==(simgl(1,3,z)+1)&&simgl(3,2,z)==(simgl
(3,3,z)+2))
390         spixel(:, :, z) = [1 1 0 1 0 1];
391     else if (simgl(1,2,z)==(simgl(3,1,z)-1)&&simgl(2,1,z)==(simgl(3,1,z)+1)&&simgl
(2,2,z)==(simgl(3,1,z)+2)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,1,z)+1))
392         spixel(:, :, z) = [1 1 0 1 1 0];
393     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(3,1,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)+2)&&simgl(2,3,z)==(simgl(3,3,z)+1)&&simgl(3,2,z)==(simgl
(1,3,z)+1))
394         spixel(:, :, z) = [1 1 0 1 1 1];
395     else if (simgl(1,2,z)==(simgl(3,1,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,1,z)+1)&&simgl(2,3,z)==(simgl(3,1,z)+2)&&simgl(3,2,z)==(simgl
(1,1,z)+1))
396         spixel(:, :, z) = [1 1 1 0 0 0];
397     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,1,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)+2)&&simgl(3,2,z)==(simgl
(1,3,z)+1))
398         spixel(:, :, z) = [1 1 1 0 0 1];
399     else if (simgl(1,2,z)==(simgl(3,1,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,1,z)+2)&&simgl(2,3,z)==(simgl(1,1,z)+1)&&simgl(3,2,z)==(simgl
(3,1,z)+1))
400         spixel(:, :, z) = [1 1 1 0 1 0];
401     else if (simgl(1,2,z)==(simgl(3,3,z)-1)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)+2)&&simgl(2,3,z)==(simgl(1,3,z)+1)&&simgl(3,2,z)==(simgl
(3,1,z)-1))
402         spixel(:, :, z) = [1 1 1 0 1 1];
403     else if (simgl(1,2,z)==(simgl(3,1,z)+2)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,1,z)-1)&&simgl(2,3,z)==(simgl(3,1,z)+1)&&simgl(3,2,z)==(simgl
(1,1,z)+1))
404         spixel(:, :, z) = [1 1 1 1 0 0];
405     else if (simgl(1,2,z)==(simgl(3,3,z)+2)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,3,z)-1)&&simgl(2,3,z)==(simgl(3,1,z)+1)&&simgl(3,2,z)==(simgl
(1,3,z)+1))
406         spixel(:, :, z) = [1 1 1 1 0 1];
407     else if (simgl(1,2,z)==(simgl(3,1,z)+2)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,1,z)+1)&&simgl(2,3,z)==(simgl(3,1,z)-1)&&simgl(3,2,z)==(simgl
(1,1,z)+1))
408         spixel(:, :, z) = [1 1 1 1 1 0];
409     else if (simgl(1,2,z)==(simgl(3,3,z)+2)&&simgl(2,1,z)==(simgl(3,3,z)+1)&&simgl
(2,2,z)==(simgl(3,1,z)+1)&&simgl(2,3,z)==(simgl(3,3,z)-1)&&simgl(3,2,z)==(simgl
(1,3,z)+1))
410         spixel(:, :, z) = [1 1 1 1 1 1];
411     end
412 end
413 end
414 end

```

```
415         end
416     end
417 end
418 end
419 end
420 end
421 end
422 end
423 end
424 end
425 end
426 end
427 end
428 end
429 end
430 end
431 end
432 end
433 end
434 end
435 end
436 end
437 end
438 end
439 end
440 end
441 end
442 end
443 end
444 end
445 end
446 end
447 end
448 end
449 end
450 end
451 end
452 end
453 end
454 end
455 end
456 end
457 end
458 end
459 end
460 end
461 end
462     end
```

```

463         end
464     end
465 end
466 end
467 end
468     end
469 end
470 end
471 end
472 end
473 end
474 end
475 end
476 end
477
478
479
480 sA= reshape(spixel,1,2,[]);
481
482 h=1;
483 q1=[sA(:, :, h) to sA(:, :, h+ 127)];
484 h=h+128;
485 q2=[sA(:, :, h) to sA(:, :, h+ 127)];
486 h=h+128;
487 q3=[sA(:, :, h) to sA(:, :, h+ 127)];
488 h=h+128;
489 q4=[sA(:, :, h) to sA(:, :, h+ 127)];
490 h=h+128;
491 q5=[sA(:, :, h) to sA(:, :, h+ 127)];
492 h=h+128;
493 q6=[sA(:, :, h) to sA(:, :, h+ 127)];
494 h=h+128;
495 q7=[sA(:, :, h) to sA(:, :, h+ 127)];
496 h=h+128;
497 q8=[sA(:, :, h) to sA(:, :, h+ 127)];
498 h=h+128;
499 q9=[sA(:, :, h) to sA(:, :, h+ 127)];
500 h=h+128;
501 q10=[sA(:, :, h) to sA(:, :, h+ 127)];
502 h=h+128;
503
504         .
505         .
506         .
507         .
508         .
509
510         .

```

```

511         .
512         .
513         .
514         .
515         upto
516 q256=[sA(:, :, h) to sA(:, :, h+ 127)];
517
518 B3=reshape(ww,[], 256)';
519 figure ,imshow(B3)
520 imsave()

```

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Results



Figure 5.1: Stego Image



Figure 5.2: Extracted Original Image

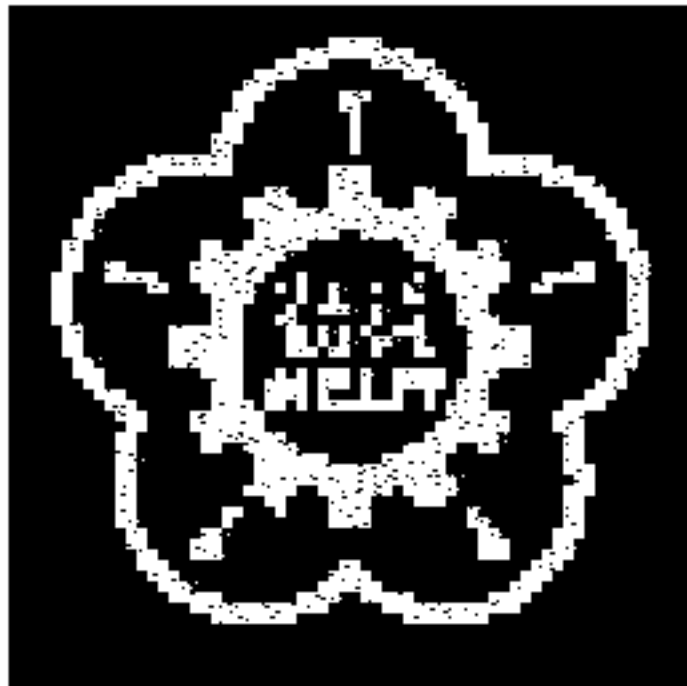


Figure 5.3: Extracted Secret Image

5.2 Discussion

In this section we discuss the experimentation work carried out for the evaluation of the proposed scheme. The proposed framework has been evaluated in terms of both subjective and objective parameters include PSNR,SSIM,BER and performed some attacks include Median Filtering,Salt and Pepper,Gaussian,Histogram Equalisation,JPEG,Low pass Filtering,Sharpening,Rotate.

Attack	Image1(peppers)	Image2(Airplane)	Image3(Lena)
Median filtering	 %BER=46.6	 %BER=68.5	 %BER=53.5
Salt and Pepper	 %BER=0.26	 %BER=0.25	 %BER=0.29
Gaussian	 %BER=39	 %BER=38.6	 %BER=37.2
Histogram Equalisation	 %BER=31.5	 %BER=31.68	 %BER=31.57
JPEG	 %BER=38.3	 %BER=33.1	 %BER=36.6
Low Pass Filtering	 %BER=43	 %BER=44.5	 %BER=43.4
Sharpening	 %BER=32.8	 %BER=33.1	 %BER=31.5
Rotate (1 degree)	 %BER=44.5	 %BER=47.1	 %BER=45.5

CHAPTER 6

CONCLUSION

6.1 Conclusion

Information or data is very crucial resource to us. Thus securing the information becomes all the more necessary. The communication media through which we send data does not provide data security, so other methods of securing data are required. Information hiding plays a very crucial role today. It provided methods for encrypting the information so that it becomes unreadable for any unintended user. Data or information is very crucial to any organization or any individual person. None of us likes our conversation being overheard as it contains the potential of being misused. Same is the case with the data of any organization or of any person. The exchange of data among two potential parties must be done in a secured method so as to avoid any tampering. Two types of threats exist during any information exchange. The unintended user who may try to overhear this conversation can either tamper with this information to change its original meaning or it can try to listen to the message with intention to decode it and use it to his/her advantage. Both these attacks violated the confidentiality and integrity of the message passed. Providing intended access and avoiding unintended access is a very challenging task. Information hiding has been since long time. In past, people used hidden pictures or invisible ink to convey secret information. The importance of data hiding techniques comes from the fact that there is no reliability over the medium through which the information is sent, in other words the medium is not secured. So, some methods are needed so that it becomes difficult for unintended user to extract the information from the message. Thus we have developed a secure algorithm utilising information hiding approach to ensure client data security.

REFERENCES

- [1] Kaw, J. A., Loan, N. A., Parah, S. A., Muhammad, K., Sheikh, J. A., Bhat, G. M. (2019). A reversible and secure patient information hiding system for IoT driven e-health. *International Journal of Information Management*, 45, 262-275.
- [2] Jung, K. H., Yoo, K. Y. (2009). Data hiding method using image interpolation. *Computer Standards Interfaces*, 31(2), 465-470.
- [3] Wang, X. T., Chang, C. C., Nguyen, T. S., Li, M. C. (2013). Reversible data hiding for high quality images exploiting interpolation and direction order mechanism. *Digital Signal Processing*, 23(2), 569-577.
- [4] Mohammad, A. A., Al-Haj, A., Farfoura, M. (2019). An improved capacity data hiding technique based on image interpolation. *Multimedia Tools and Applications*, 78(6), 7181-7205.
- [5] Tseng, Y. C., Chen, Y. Y., Pan, H. K. (2002). A secure data hiding scheme for binary images. *IEEE transactions on communications*, 50(8), 1227-1231.
- [6] Chen, Y. Q., Sun, W. J., Li, L. Y., Chang, C. C., Wang, X. (2020). An efficient general data hiding scheme based on image interpolation. *Journal of Information Security and Applications*, 54, 102584.
- [7] Mandal, P. C., Mukherjee, I., Chatterji, B. N. (2021). High capacity reversible and secured data hiding in images using interpolation and difference expansion technique. *Multimedia Tools and Applications*, 80(3), 3623-3644.
- [8] Meikap, S., Jana, B. (2018). Directional PVO for reversible data hiding scheme with image interpolation. *Multimedia Tools and Applications*, 77(23), 31281-31311.
- [9] E. Khosravi, M. R., Rostami, H., Samadi, S. (2018). Enhancing the binary watermark-based data hiding scheme using an interpolation-based approach for optical remote sensing images. *International Journal of Agricultural and Environmental Information Systems (IJAEIS)*, 9(2), 53-71.
- [10] Govind, P. S., Sajila, M. K., Varghese, B. M. (2016). A two stage data hiding scheme with high capacity based on interpolation and difference expansion. *Procedia Technology*, 24, 1311-1316.
- [11] Malik, A., Sikka, G., Verma, H. K. (2017). Image interpolation based high capacity reversible data hiding scheme. *Multimedia Tools and Applications*, 76(22), 24107-24123.

- [12] Govind, P. S., Judy, M. V. (2021). A secure framework for remote diagnosis in health care: A high capacity reversible data hiding technique for medical images. *Computers Electrical Engineering*, 89, 106933.
- [13] Xiong, X., Wang, L., Li, Z., Ye, C., Chen, Y., Fan, M., Zhu, Y. (2022). An adaptive high capacity reversible data hiding algorithm in interpolation domain. *Signal Processing*, 194, 108458.
- [14] Lu, T. C. (2018). Interpolation-based hiding scheme using the modulus function and re-encoding strategy. *Signal Processing*, 142, 244-259.
- [15] Mansour, R. F., Parah, S. A. (2021). Reversible data hiding for electronic patient information security for telemedicine applications. *Arabian Journal for Science and Engineering*, 46(9), 9129-9144.